

N 诺考研系列

计算机考研复试高分指南



2023 版

noobdream.com

N 诺课程教研团队

2022. 12. 20 更新

写在前面的话

同学们通过初试就打败了绝大多数的对手，只需要再通过复试环节就可以顺利上岸了。行百里者半九十，大家切不可在最后时刻掉以轻心。

前几年因为疫情的原因，各个学校的复试操作五花八门，甚至由于线上操作的原因出现过规模较大的作弊行为，所以我们一直没有推出 N 诺的《计算机考研复试高分指南》书籍。

随着防疫政策的变化，核酸检测和健康码不再成为出行的障碍。如不出意外，我们预测 23 年 3 月左右的复试将会在线下进行，恢复成原本的线下复试流程。

复试主要分为以下三个重要环节：

1、专业能力测试

为什么初试有专业课考试，复试还要进行专业能力测试呢？主要基于两个方面的原因，一个原因是初试的专业课考试科目覆盖面小，不足以确定考生的专业能力是否全面，所以有的学校在复试中会进行一些专业课科目的笔试。另一个原因是笔试不足以确定学生的实际编程能力，曾经有笔试高分的同学连简单的代码都写不出来，甚至连 IDE 都不会用。所以很多学校都会在复试通过上机编程考试环节来检测考生的实际编程能力。而且机试很难几天之内速成，所以复试有机试的同学一定要提前练习。

2、英语口语测试

英语口语也是一个必要的测试环节，不过初试笔试中已经考察了英语科目，所以这个环节的分数比重一般较小，大多数同学差距都不大，只需要花一点时间准备就可以。

3、综合面试

一般来说，很难通过一二十分钟的面试确定一个陌生人的真实能力，因为面试官问你的问题可能刚好是你擅长的或者不擅长的。对大多数同学来说，肯定会有不少知识盲区，所以掌控面试节奏，学会引导面试官问自己提前准备好的问题是至关重要的。

其他一些如：同等学力加试、体检、政审等环节不在本书的讨论范围之内，因为对几乎所有同学来说，这些环节不会有问题，按学校通知规定走正常流程即可。

关于 N 诺

N 诺是全国最大的计算机考研在线学习平台。N 诺致力于为同学们提供一个良好的网络学习环境，一个与大佬们随时随地交流的舒适空间。如果你不知道 N 诺，那么你已经输在起跑线上了，因为 N 诺是 - 计算机学习考研必备神器。

N 诺整理了**计算机考研报考指南**，帮助你了解考研的点点滴滴赢在起跑线上。

<http://www.noobdream.com/media/upload/2020/01/29/guide.pdf>

在 N 诺，你可以查询各个院校的**考研信息**，包括分数线、录取人数、考试大纲、导师信息、经验交流等信息，应有尽有。

<http://www.noobdream.com/schoollist/>

在 N 诺，你可以尽情的刷各个科目的**题库**，还可以将题目加入**错题本**方便以后复习，也可以写下自己的**学习笔记**，记录考研过程中跌宕起伏。

<http://www.noobdream.com/Practice/index/>

在 N 诺，你可以在**讨论区**里发表你的问题或感想，与全国几百万考研 er 分享你的喜怒哀乐。

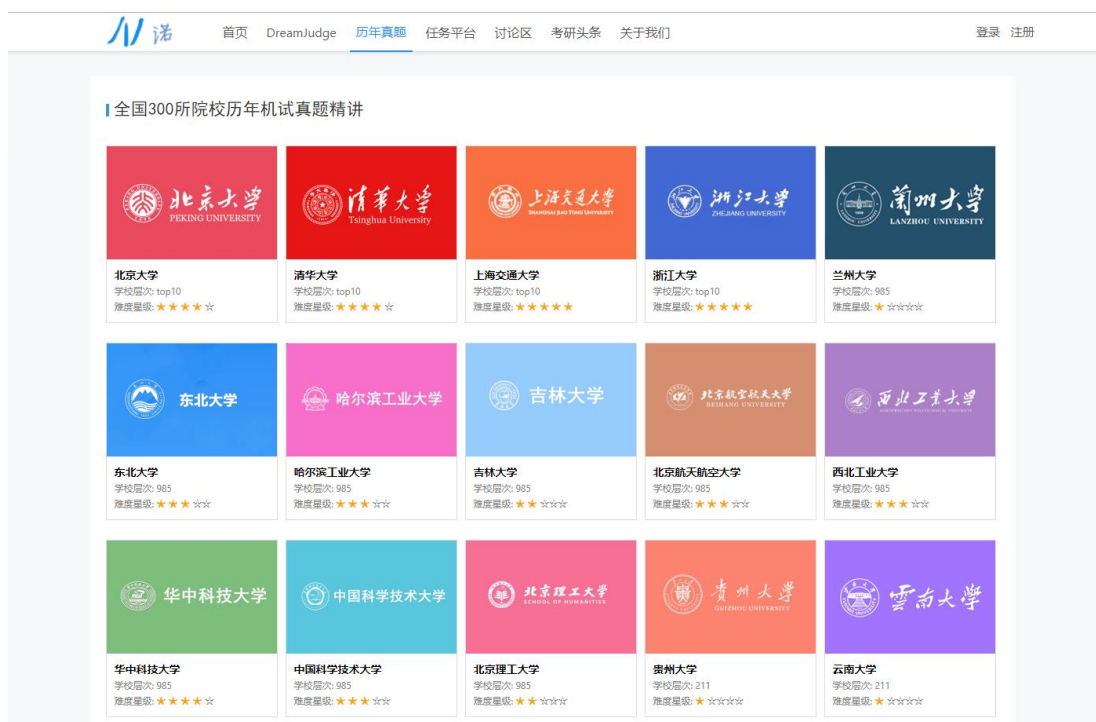
<http://www.noobdream.com/forum/0/>

在 N 诺，你可以将你不用**的书籍或资料**放到**二手交易市场**，既能帮助他人，还能为自己省下一笔当初买书买资料的费用。

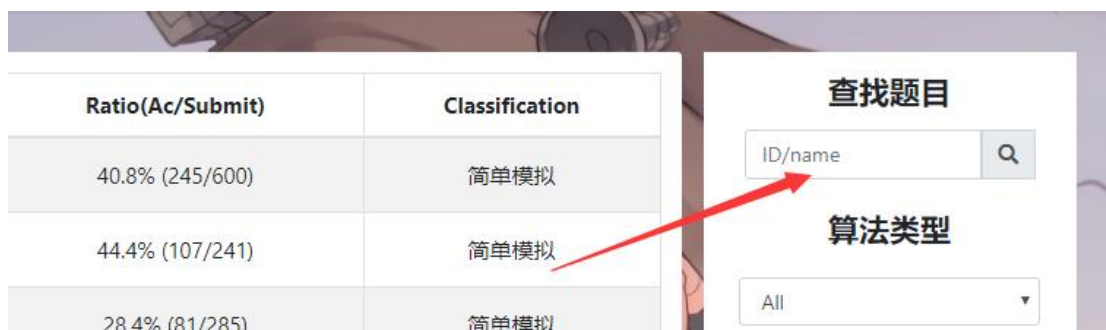
<http://www.noobdream.com/Task/tasklist/>

如何使用本书？

访问 N 诺平台 (www.noobdream.com) 的历年真题, 即可查询到全国各个学校的历年机试真题。



如下图输入课后习题的编号即可搜索到题目



小技巧: N 诺是可以随意更换皮肤的哟, 在右上角头像旁边。

目录

写在前面的话	2
关于 N 诺	3
如何使用本书?	4
目录	5
第一章 专业能力测试	6
1.1 机试	6
1.1.1 机试学习资料	6
1.1.2 机试分析	7
1.1.3 IDE 的选择与评测结果	9
1.1.4 DreamJudge 的使用	10
1.2 专业课笔试	11
1.2.1 代码人工打分	11
1.2.2 考察其他专业课	13
第二章 英语口语能力测试	14
2.1 英语口语	14
2.1.1 考察形式	14
2.2.2 常见问题	15
第三章 综合面试	16
3.1 需要做哪些提前准备?	16
3.2 面试技巧总结	17
3.3 常考专业问题汇总	18
3.3.1 C/C++ 语言	18
3.3.2 Java 语言	30
3.3.3 数据结构	44
3.3.4 操作系统	49
3.3.5 计算机网络	66
3.3.6 数据库	81
3.3.7 软件测试	92
3.3.8 机器学习	97
3.3.9 设计模式	100
第四章 常见问题答疑	102
4.1 考研流程	102
4.2 复试流程	105
4.3 复试是否需要报培训班?	106
N 诺考研系列图书	107

第一章 专业能力测试

1.1 机试

上机考试很多同学来说是一道坎，分数差距极大。学的好的同学能拿满分，学的差的同学一道题都做不出来也不奇怪，所以编程能力差的同学一定要在复试前的两个月时间里尽力提高自己的机试水平。

机试其实很简单，以我们多年的经验来看，即使是零基础的同学花一个月时间有效学习也能在机试中取得高分的成绩。

很多同学写代码最大的问题是，代码有问题不知道哪里错了，然后又找不到人问，导致花了大量时间在改错上面，产生了很多无效学习时间。

学习效率的高低直接决定了机试成绩的高低，遇到问题要尽早解决，不要在一道题上浪费太多的时间。

1.1.1 机试学习资料

参考书：《计算机考研机试攻略 高分篇》、《计算机考研机试攻略 满分篇》

获取方式 1：加入 N 诺官方群（[962884669](#)）在群文件中下载

获取方式 2：关注 N 诺公众号（[N 诺计算机考研](#)）回复**考研机试**即可下载

N 诺小程序:



B 站免费学习视频

<https://www.bilibili.com/video/BV1AJ411Y7Fm>

<https://www.bilibili.com/video/BV1G7411u7o5>

1.1.2 机试分析

首先我们来看一下机试是怎样的一种考核模式

全国所有院校的机试都大同小异，大部分有自己 OJ (Online Judge 也就是在线代码测评平台) 的学校都会采用 OJ 上做题的方式来进行考核。这种考核方式的好处是公开透明，机器进行判题并给分，就像 N 诺的 DreamJudge 一样。没有 OJ 的学校只能人工进行判题，人工判题的话，一方面是主观性比较强，可能还会对其他方面进行考量，这个就需要自己去了解了。总的来说，不论是 OJ 判题还是人工判题，代码都要能通过测试用例才能得到分数。

针对机试我们应该怎么去训练提升自己呢

首先，一定要做题，在 N 诺上做题，不要自己埋头看书，在不断做题的过程中才能提升自己。如果非要用一个量化的标准来衡量的话，至少要在 N 诺上做够 100 题 (也就是达到砖石

II 以上段位)，才能保证你机试达到自己满意的成绩。题量是很关键的，看懂的题再多，都不如自己实际敲代码去解决问题更稳妥。

解题速度也是很重要的

其实解题速度和题量是正相关的，相信题量足够的同学，解题的速度都不会太慢。机试一般 2-3 个小时左右，要解决 5-8 道题。平均下来一道题最多半个小时，从读题、分析题意、思考解法、敲代码、调试、测试数据到最后提交这整个流程下来，如果你平时训练的少，读题慢、理解题意慢、思考解法慢、敲代码慢、调试慢，这样一算下来，一道简单的题都可能要一个小时才能写出来，说不定题目还有坑点，再慢慢调试，基本上就凉了。

多打比赛也是很重要的

很多同学平时做的题很多，解题速度也挺快，但是一到比赛或者考试的时候就会卡题，在压力的情况下发挥失常比比皆是，所以平时就要锻炼自己的抗压能力。

N 诺上除了每个周定期举办的小白赛，还特别为大家准备了**考研机试冲刺八套卷**。

在本书的最后，你可以看到关于考研机试冲刺八套卷的详细信息。通过这八套卷的练习，相信会让你的水平产生一个脱胎换骨的变化。

准备好模板是至关重要的

一般来说，机试都可以带书和纸质资料进入考场。所以提前把那些函数的用法和算法的模板准备好是很重要的，一方面是增加自己的信心，万一没记住还可以翻开来看一下。另外说不定考到原题或者类似的题，就可以直接秒杀了。

特别提醒：N 诺的《计算机考研机试攻略》默认读者是会 C 语言的基本语法的，比如 if 语句、for 语句等等。

C 语言基本语法还未掌握的同学建议学习 N 诺的 C 语言快速入门课程(3 天学会 C 语言不是梦)

地址：<http://www.noobdream.com/Major/majorinfo/1/>

1.1.3 IDE 的选择与评测结果

建议选择 **CodeBlocks** 作为平时敲代码练习的 IDE

下载地址: <http://www.noobdream.com/Major/article/1/>

常见做题结果反馈

Accepted: 答案正确, 恭喜你正确通过了这道题目。

Wrong Answer: 答案错误, 出现这个错误的原因一般是你的程序实现或思路出现了问题, 或者数据范围边界没有考虑到。

Runtime Error: 运行时错误, 出现这个错误的原因一般是数组越界或者递归过深导致栈溢出。

Presentation Error: 输出格式错误, 出现这个错误的原因一般是末尾多了或少了空格, 多了或少了换行

Time Limit Exceeded: 程序运行超时, 出现这个错误的原因一般是你的算法不够优秀, 导致程序运行时间过长。

Memory Limit Exceeded: 运行内存超限, 出现这个错误的原因一般是你的程序申请太大了空间, 超过了题目规定的空间大小。

Compile Error: 编译错误, 这个不用说了吧, 就是你的代码存在语法错误, 检查一下是不是选择错误的语言提交了。

Output Limit Exceeded: 输出超限, 程序输出过多的内容, 一般是循环出了问题导致多次输出或者是调试信息忘记删除了。

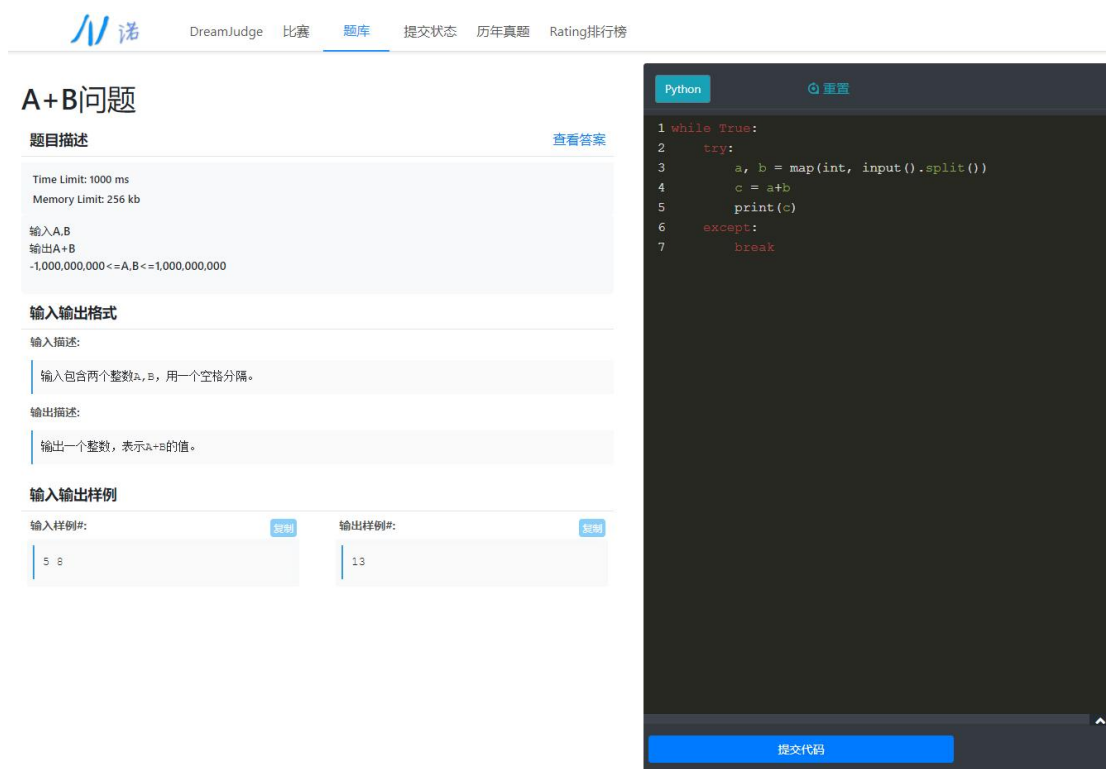
Submitting: 提交中, 请等待题目结果的返回, 由于判题机有性能差异, 所以返回结果的速度也不一样, N 诺上做题一般瞬间就能出结果。

以上几种结果就是评判系统可能会返回的几种常见结果。若返回 Accept, 那么你就可以拿到该题所有分数, 如果返回其他结果, 则要看你报考学校的考试规则, 是根据通过测试点的百分比给分还是只要不是 AC 就得 0 分。

1.1.4 DreamJudge 的使用

DreamJudge 是一个在线代码测评的平台，可以很方便的检验自身的学习情况。使用 DreamJudge 的方法很简单，通过百度搜索 N 诺或者在浏览器中输入网址 www.noobdream.com 进入 N 诺然后点击网站导航上方的 DreamJudge 就可以进去啦。

做题页面如下：



The screenshot shows the DreamJudge website interface. The top navigation bar includes links for DreamJudge, 比赛 (Contests), 题库 (Problem Set), 提交状态 (Submission Status), 历年真题 (Past Exams), and Rating排行榜 (Rating Ranking). The main content area is titled 'A+B问题' (A+B Problem). Under '题目描述' (Problem Description), it specifies a Time Limit of 1000 ms and a Memory Limit of 256 kb. The input is two integers A and B, and the output is their sum A+B. The constraints are $-1,000,000,000 \leq A, B \leq 1,000,000,000$. The '输入输出格式' (Input/Output Format) section states that the input consists of two integers A and B separated by a space, and the output is a single integer representing A+B. The '输入输出样例' (Input/Output Examples) section shows an input of '5 8' and an output of '13'. On the right side, there is a code editor for Python with the following code:

```
1 while True:
2     try:
3         a, b = map(int, input().split())
4         c = a+b
5         print(c)
6     except:
7         break
```

At the bottom of the code editor, there is a blue button labeled '提交代码' (Submit Code).

首先要登录我们的 N 诺账号，然后开始做题。如果没有账号，右上角点击注册，然后注册一个账号就可以了。

然后将代码粘贴到右边的输入框里，在上面选择使用哪种语言提交，C/C++/Java/Python，建议选择 C++ 提交，因为 C++ 可以编译 C 语言代码。我们一般写代码为了方便，都会使用一点 C++ 的特性来帮助我们快速解决一道题目。如果代码里含有 C++ 的特性却选择了 C 语言提交的话，会返回编译错误的提示信息。

1.2 专业课笔试

有的学校由于生源不是很好, 如果采用机考的形式对专业能力进行检测, 可能会有很多同学都是零分, 特别是跨专业考研的同学。

所以为了降低难度, 就出现了两种类似初试专业课考试的形式的考察方式。

1.2.1 代码人工打分

我们前面知道机考可以通过 OJ 平台自动判断对错, 如果做错了或者不会做就是 0 分, 是没有过程分的。有的学校通过人工改代码的方式, 虽然你的代码没有写完整, 或者写的不对, 依然可以给你较多的过程分。

那么如何才能在尽量骗到更高的过程分呢?

关键在于提前准备好一套唬人的模板代码, 让改卷的老师拿不准你是真的菜还是状态不好没做出来。

例如把输入输出和计算过程都写成一个函数, 以一个最简单的 A+B 代码为例

```
#include <stdio.h>
```

```
int a, b;
```

```
void Input() {
```

```
    scanf("%d%d", &a, &b);
```

```
}
```

```
void Output(int ans) {
```

```
    printf("%d\n", ans);
```

```
}
```

```
void Calc() {
```

```
    int sum = a + b;
```

```
    Output(sum);  
}
```

```
int main() {  
    Input();  
    Calc();  
    return 0;  
}
```

我们还可以更进一步，用一个结构体或者类来对函数进行封装，代码可以修改为：

```
#include <stdio.h>
```

```
struct Add {  
    int a, b;  
    int sum;  
    void Input() {  
        scanf("%d%d", &a, &b);  
    }  
};
```

```
    void Output() {  
        printf("%d\n", sum);  
    }
```

```
    void Calc() {  
        sum = a + b;  
    }  
};
```

```
int main() {
```

```
Add exp;  
exp.Input();  
exp.Calc();  
exp.Output();  
return 0;  
}
```

同学们做其他题的时候也可以参照上面的过程, 将输入、输出以及计算的过程进行分离。就算最终计算的结果不对, 也会比把所有代码全部写在 main 函数里要好。

还有一个问题就是, 要尽量注意代码的排版的格式要让人赏心悦目, 注意缩进和对齐, 代码不要挤在一坨, 和写字一样, 我们都知道字写的好的同学, 分数往往都会高一点, 代码同理。



1.2.2 考察其他专业课

其他专业课的范围就比较广了, 有的学校会考数据库、软件工程、算法分析等等课程。

对于每一门课都是一本书的内容, 无法在这里一一给同学们讲解。

建议同学们去看看对应课程考纲指定的书籍和期末试卷进行练习即可。

第二章 英语口语能力测试

2.1 英语口语

英语口语的分数占比比较小，对于大多数同学来说只需要提前准备练习一下都没有问题，而且分数差距也比较小，建议同学们合理分配时间来准备这部分的内容。



2.1.1 考察形式

英语口语的考察形式根据不同学校有所区别，一般是先自我介绍，然后根据老师提出的问题进行回答即可。

自我介绍部分同学们提前准备好内容然后背诵熟悉即可。

需要注意的是，有的老师会直接根据自我介绍的内容来问问题，比如你自我介绍中说到你是某个地方的人，老师就可能会问你你的家乡怎么样，有什么好吃的好玩的之类的。

2.2.2 常见问题

问题分为两类，一类是聊天问题，另一类是专业问题。

聊天类问题就很多，比如问你的爱好、家乡、为什么考研、大学怎么样等等日常问题。这类问题难度不大，可以提前准备一些常见的回答，口语不行的同学可以多读读。

专业问题一般比较开放，比如你觉得 XX 方向怎么样，主要考察对专业词汇的积累，如果你没有专业词汇积累，可能连问题都听不懂，如果听不懂问题不要不懂装懂，应该准备一套话来应对这种情况，因为这个环节重点考察的是口语而不是专业能力，所以委婉的向老师表达某个专业名词不知道什么意思，或者对某个前沿方向不太了解无法回答表示抱歉，而不是直接说我不知道。

核心就是要有自信，流畅的表达展现自己的口语能力。



第三章 综合面试

3.1 需要做哪些提前准备?

个人简历准备

首先要准备一份个人简历, 可以多打印几份, 方便老师快速了解你的优点。

那么简历应该写哪些内容呢?

很明显要展示自己的优势, 缺点之类的就不要写进去了。

最常见的就是在大学期间获得哪些奖项, 特别是专业相关的比如大学生程序设计竞赛之类的奖项。

其次就是如果大学期间成绩好绩点高, 也可以写进去。

一些无关紧要的内容就不要写了, 比如身高、体重、兴趣爱好等等。因为时间有限, 写的太多反而无法让人快速 **get** 到你的亮点。

项目准备

如果大学期间有一个不错的项目其实是可以重点介绍的, 对于科班的同学, 毕业论文的项目是重点, 要着重去准备。

跨专业的同学时间上来不及从头写一个项目了, 可以直接从网上 (比如 **github**) 找一个项目, 然后梳理清楚项目的架构和技术难点, 总结总结就可以当做是自己做过的项目了。

简单点说, 就是拿来主义, 只要把别人的项目逻辑分析清楚了, 知道怎么实现的, 就可以说是自己做过的, 因为不可能在面试中真的让你写代码实现。

所以自信很重要, 如果不自信, 就算是自己写的项目, 老师也可能认为你是网上看来的。

3.2 面试技巧总结

1、在简历上做一个引导

在词汇上做好区分，比如熟悉 Java，了解 python，精通 c 语言

这样表达，对自己的掌握程度能有个区分，也好让面试官有个着重去问，python 本来写的也只是了解，自然就不会多问你深入的一些东西了。

2、在面试过程中做一个引导

面试过程中尽量引导到自己熟知的一个领域，比如问到你说一下 DNS 寻址，然后你简单回答（甚至这步也可以省略）之后，可以说一句，自己对这块可能不是特别熟悉，对计算机网络中的运输层比较熟悉，如果有具体的，甚至可以再加一句，比如 TCP 和 UDP

这样的话你可以把整个面试过程往你熟知的地方引导，也能更倾向于体现出你的优势而不是劣势，但是此方法仅限于掌握合适的度，比如有的知识点是必会的，而你想往别处引就有点说不过去了，比如让你说几个 c++ 的关键字，你一个也说不上来，那可能就真的没辙了。

3、在自我介绍中做一个引导

一般面试的开头都会有一个自我介绍，在这个位置你也可以尽情的为自己的优势方面去引导。

4、面试过程中展示出自信

面试过程中的态度也要掌握好，不要自卑，也不要傲娇，自信的回答出每个问题，尤其遇到不会的问题，要么做一些引导，实在不能引导也可以先打打擦边球，和面试官交流一下问题，看起来像是没听懂题意，这个过程也可以再自己思考一下，如果觉得这个过程可以免了的话也直接表明一下这个地方不太熟悉或者还没有掌握好，千万不要强行回答。

3.3 常考专业问题汇总

3.3.1 C/C++语言

1、简述 C++语言的特点

C++在 C 语言基础上引入了面对对象的机制，同时也兼容 C 语言。

C++有三大特性（1）封装。（2）继承。（3）多态；

C++语言编写出的程序结构清晰、易于扩充，程序可读性好。

C++生成的代码质量高，运行效率高，仅比汇编语言慢 10%~20%；

C++更加安全，增加了 `const` 常量、引用、四类 `cast` 转换（`static_cast`、`dynamic_cast`、`const_cast`、`reinterpret_cast`）、智能指针、`try-catch` 等等；

C++可复用性高，C++引入了模板的概念，后面在此基础上，实现了方便开发的标准模板库 STL（Standard Template Library）。

同时，C++是不断在发展的语言。C++后续版本更是发展了不少新特性，如 C++11 中引入了 `nullptr`、`auto` 变量、Lambda 匿名函数、右值引用、智能指针。

2、C 语言和 C++的区别

C 语言是 C++的子集，C++可以很好兼容 C 语言。但是 C++又有很多新特性，如引用、智能指针、`auto` 变量等。

C++是面对对象的编程语言；C 语言是面对过程的编程语言。

C 语言有一些不安全的语言特性，如指针使用的潜在危险、强制转换的不确定性、内存泄露等。而 C++对此增加了不少新特性来改善安全性，如 `const` 常量、引用、`cast` 转换、智能指针、`try-catch` 等等；

C++可复用性高，C++引入了模板的概念，后面在此基础上，实现了方便开发的标准模板库 STL。

C++的 STL 库相对于 C 语言的函数库更灵活、更通用。

3、C++中 `struct` 和 `class` 的区别

`struct` 一般用于描述一个数据结构集合，而 `class` 是对一个对象数据的封装；

`struct` 中默认的控制访问权限是 `public` 的，而 `class` 中默认的控制访问权限是 `private` 的，例

如:

```
struct A{ int iNum; // 默认访问控制权限是 public } class B{ int iNum; // 默认访问控制权限是 private }
```

在继承关系中, `struct` 默认是公有继承, 而 `class` 是私有继承;

`class` 关键字可以用于定义模板参数, 就像 `typename`, 而 `struct` 不能用于定义模板参数, 例如:

```
template<typename T, typename Y> // 可以把 typename 换成 class int Func(const T& t, const Y& y) { //TODO }
```

4、include 头文件的顺序以及双引号""和尖括号<>的区别

区别:

- (1) 尖括号<>的头文件是系统文件, 双引号""的头文件是自定义文件。
- (2) 编译器预处理阶段查找头文件的路径不一样。

查找路径:

- (1) 使用尖括号<>的头文件的查找路径: 编译器设置的头文件路径-->系统变量。
- (2) 使用双引号""的头文件的查找路径: 当前头文件目录-->编译器设置的头文件路径-->系统变量。

5、简述 C++从代码到可执行二进制文件的过程

C++和 C 语言类似, 一个 C++程序从源码到执行文件, 有四个过程, 预编译、编译、汇编、链接。

答案解析

预编译: 这个过程主要的处理操作如下:

- (1) 将所有的`#define` 删除, 并且展开所有的宏定义
- (2) 处理所有的条件预编译指令, 如`#if`、`#ifdef`
- (3) 处理`#include` 预编译指令, 将被包含的文件插入到该预编译指令的位置。
- (4) 过滤所有的注释
- (5) 添加行号和文件名标识。

编译: 这个过程主要的处理操作如下:

- (1) 词法分析：将源代码的字符序列分割成一系列的记号。
- (2) 语法分析：对记号进行语法分析，产生语法树。
- (3) 语义分析：判断表达式是否有意义。
- (4) 代码优化：
- (5) 目标代码生成：生成汇编代码。
- (6) 目标代码优化：

汇编：这个过程主要是将汇编代码转变成机器可以执行的指令。

链接：将不同的源文件产生的目标文件进行链接，从而形成一个可以执行的程序。

链接分为静态链接和动态链接。

静态链接，是在链接的时候就已经把要调用的函数或者过程链接到了生成的可执行文件中，就算你在去把静态库删除也不会影响可执行程序的执行；生成的静态链接库，Windows 下以.lib 为后缀，Linux 下以.a 为后缀。

而动态链接，是在链接的时候没有把调用的函数代码链接进去，而是在执行的过程中，再去找要链接的函数，生成的可执行文件中没有函数代码，只包含函数的重定位信息，所以当你删除动态库时，可执行程序就不能运行。生成的动态链接库，Windows 下以.dll 为后缀，Linux 下以.so 为后缀。

6、static 关键字的作用

定义全局静态变量和局部静态变量：在变量前面加上 **static** 关键字。初始化的静态变量会在数据段分配内存，未初始化的静态变量会在 BSS 段分配内存。直到程序结束，静态变量始终会维持前值。只不过全局静态变量和局部静态变量的作用域不一样；

定义静态函数：在函数返回类型前加上 **static** 关键字，函数即被定义为静态函数。静态函数只能在本源文件中使用；

在变量类型前加上 **static** 关键字，变量即被定义为静态变量。静态变量只能在本源文件中使用；

//示例 `static int a; static void func();`

在 c++ 中，**static** 关键字可以用于定义类中的静态成员变量：使用静态数据成员，它既可以被当成全局变量那样去存储，但又被隐藏在类的内部。类中的 **static** 静态数据成员拥有一块单独的存储区，而不管创建了多少个该类的对象。所有这些对象的静态数据成员都共享这一块静

态存储空间。

在 c++ 中, **static** 关键字可以用于定义类中的静态成员函数: 与静态成员变量类似, 类里面同样可以定义静态成员函数。只需要在函数前加上关键字 **static** 即可。如静态成员函数也是类的一部分, 而不是对象的一部分。所有这些对象的静态数据成员都共享这一块静态存储空间。

7、数组和指针的区别

概念:

- (1) 数组: 数组是用于储存多个相同类型数据的集合。 数组名是首元素的地址。
- (2) 指针: 指针相当于一个变量, 但是它和不同变量不一样, 它存放的是其它变量在内存中的地址。 指针名指向了内存的首地址。

区别:

- (1) 赋值: 同类型指针变量可以相互赋值; 数组不行, 只能一个一个元素的赋值或拷贝
- (2) 存储方式:

数组: 数组在内存中是连续存放的, 开辟一块连续的内存空间。数组是根据数组的下标进行访问的, 数组的存储空间, 不是在静态区就是在栈上。

指针: 指针很灵活, 它可以指向任意类型的数据。指针的类型说明了它所指向地址空间的内存。由于指针本身就是一个变量, 再加上它所存放的也是变量, 所以指针的存储空间不能确定。

- (3) 求 **sizeof**:

数组所占存储空间的内存大小: **sizeof (数组名) / sizeof (数据类型)**

在 32 位平台下, 无论指针的类型是什么, **sizeof (指针名)** 都是 4, 在 64 位平台下, 无论指针的类型是什么, **sizeof (指针名)** 都是 8。

- (4) 初始化:

```
// 数组 int a[5] = { 0 }; char b[] = "Hello"; // 按字符串初始化, 大小为 6 char c[] = { 'H','e','l','l','o','\0' }; // 按字符初始化 int* arr = new int[10]; // 动态创建一维数组 // 指针 // 指向对象的指针 int* p = new int(0); delete p; // 指向数组的指针 int* p1 = new int[10]; delete[] p1; // 指向类的指针: string* p2 = new string; delete p2; // 指向指针的指针 (二级指针) int** pp = &p; **pp = 10;
```

- (5) 指针操作:

数组名的指针操作:

```
int a[3][4];    int (*p)[4]; //该语句是定义一个数组指针, 指向含 4 个元素的一维数组 p = a;  
//将该二维数组的首地址赋给 p, 也就是 a[0]或&a[0][0] p++; //该语句执行过后, 也  
就是 p=p+1; p 跨过行 a[0][]指向了行 a[1][] //所以数组指针也称指向一维数组  
的指针, 亦称行指针。 //访问数组中第 i 行 j 列的一个元素, 有几种操作方式: /*(p[i]+j)、  
*(*(p+i)+j)、(*(p+i))[j]、p[i][j]。其中, 优先级: ()>[]>*. //这几种操作方式都是合法的。
```

指针变量的数据操作:

```
char *str = "hello,douya!";  
str[2] = 'a';  
*(str+2) = 'b';  
//这两种操作方式都是合法的。
```

8、静态变量什么时候初始化?

对于 C 语言的全局和静态变量, 初始化发生在任何代码执行之前, 属于编译期初始化。
而 C++标准规定: 全局或静态对象当且仅当对象首次用到时才进行构造。

9、什么是野指针, 怎么产生的, 如何避免?

概念: 野指针就是指针指向的位置是不可知的(随机的、不正确的、没有明确限制的)
产生原因: 释放内存后指针不及时置空(野指针), 依然指向了该内存, 那么可能出现非法访问的错误。这些我们都要注意避免。

避免办法:

- (1) 初始化置 NULL
- (2) 申请内存后判空
- (3) 指针释放后置 NULL
- (4) 使用智能指针

10、静态局部变量, 全局变量, 局部变量的特点, 以及使用场景

首先从作用域考虑: C++里作用域可分为 6 种: 全局, 局部, 类, 语句, 命名空间和文件作用域。

1.全局变量：全局作用域，可以通过 `extern` 作用于其他非定义的源文件。

静态全局变量：全局作用域+文件作用域，所以无法在其他文件中使用。

2.局部变量：局部作用域，比如函数的参数，函数内的局部变量等等。

3.静态局部变量：局部作用域，只被初始化一次，直到程序结束。

从所在空间考虑：除了局部变量在栈上外，其他都在静态存储区。因为静态变量都在静态存储区，所以下次调用函数的时候还是能取到原来的值。

4.生命周期：局部变量在栈上，出了作用域就回收内存；而全局变量、静态全局变量、静态局部变量都在静态存储区，直到程序结束才会回收内存。

使用场景：从它们各自特点就可以看出各自的应用场景，不再赘述。

11、内联函数和宏函数的区别

区别：

1. 宏定义不是函数，但是使用起来像函数。预处理器用复制宏代码的方式代替函数的调用，省去了函数压栈退栈过程，提高了效率；而内联函数本质上是一个函数，内联函数一般用于函数体的代码比较简单的函数，不能包含复杂的控制语句，`while`、`switch`，并且内联函数本身不能直接调用自身。

2. 宏函数是在预编译的时候把所有的宏名用宏体来替换，简单的说就是字符串替换；而内联函数则是在编译的时候进行代码插入，编译器会在每处调用内联函数的地方直接把内联函数的内容展开，这样可以省去函数的调用的开销，提高效率

3. 宏定义是没有类型检查的，无论对还是错都是直接替换；而内联函数在编译的时候会进行类型的检查，内联函数满足函数的性质，比如有返回值、参数列表等

12、new 和 malloc 的区别，各自底层实现原理。

`new` 是操作符，而 `malloc` 是函数。

`new` 在调用的时候先分配内存，在调用构造函数，释放的时候调用析构函数；而 `malloc` 没有构造函数和析构函数。

`malloc` 需要给定申请内存的大小，返回的指针需要强转；`new` 会调用构造函数，不用指定内存的大小，返回指针不用强转。

`new` 可以被重载；`malloc` 不行

new 分配内存更直接和安全。

new 发生错误抛出异常，malloc 返回 null

13、const 和 define 的区别。

const 用于定义常量；而 define 用于定义宏，而宏也可以用于定义常量。都用于常量定义时，它们的区别有：

const 生效于编译的阶段；define 生效于预处理阶段。

const 定义的常量，在 C 语言中是存储在内存中、需要额外的内存空间的；define 定义的常量，运行时是直接的操作数，并不会存放在内存中。

const 定义的常量是带类型的；define 定义的常量不带类型。因此 define 定义的常量不利于类型检查。

14、使用指针需要注意什么？

定义指针时，先初始化为 NULL。

用 malloc 或 new 申请内存之后，应该立即检查指针值是否为 NULL。防止使用指针值为 NULL 的内存。

不要忘记为数组和动态内存赋初值。防止将未被初始化的内存作为右值使用。

避免数字或指针的下标越界，特别要当心发生“多 1”或者“少 1”操作

动态内存的申请与释放必须配对，防止内存泄漏

用 free 或 delete 释放了内存之后，立即将指针设置为 NULL，防止“野指针”

15、内联函数和函数的区别，内联函数的作用。

内联函数比普通函数多了关键字 inline

内联函数避免了函数调用的开销；普通函数有调用的开销

普通函数在被调用的时候，需要寻址（函数入口地址）；内联函数不需要寻址。

内联函数有一定的限制，内联函数体要求代码简单，不能包含复杂的结构控制语句；普通函数没有这个要求。

内联函数的作用：内联函数在调用时，是将调用表达式用内联函数体来替换。避免函数调用的

开销。

16、C++有几种传值方式，之间的区别是什么？

传参方式有这三种：值传递、引用传递、指针传递

值传递：形参即使在函数体内值发生变化，也不会影响实参的值；

引用传递：形参在函数体内值发生变化，会影响实参的值；

指针传递：在指针指向没有发生改变的前提下，形参在函数体内值发生变化，会影响实参的值；

17、简述一下堆和栈的区别

区别：

1.堆栈空间分配不同。栈由操作系统自动分配释放，存放函数的参数值，局部变量的值等；堆一般由程序员分配释放。

2.堆栈缓存方式不同。栈使用的是一级缓存，它们通常都是被调用时处于存储空间中，调用完毕立即释放；堆则是存放在二级缓存中，速度要慢些。

3.堆栈数据结构不同。堆类似数组结构；栈类似栈结构，先进后出。

18、简述 C++的内存管理

1.内存分配方式：

在 C++中，内存分成 5 个区，他们分别是堆、栈、自由存储区、全局/静态存储区和常量存储区。

栈，在执行函数时，函数内局部变量的存储单元都可以在栈上创建，函数执行结束时这些存储单元自动被释放。

堆，就是那些由 new 分配的内存块，一般一个 new 就要对应一个 delete。

自由存储区，就是那些由 malloc 等分配的内存块，和堆是十分相似的，不过是用 free 来结束自己的生命。

全局/静态存储区，全局变量和静态变量被分配到同一块内存中

常量存储区，这是一块比较特殊的存储区，里面存放的是常量，不允许修改。

2.常见的内存错误及其对策：

(1) 内存分配未成功，却使用了它。

- (2) 内存分配虽然成功, 但是尚未初始化就引用它。
- (3) 内存分配成功并且已经初始化, 但操作越过了内存的边界。
- (4) 忘记了释放内存, 造成内存泄露。
- (5) 释放了内存却继续使用它。

对策:

- (1) 定义指针时, 先初始化为 `NULL`。
- (2) 用 `malloc` 或 `new` 申请内存之后, 应该立即检查指针值是否为 `NULL`。防止使用指针值为 `NULL` 的内存。
- (3) 不要忘记为数组和动态内存赋初值。防止将未被初始化的内存作为右值使用。
- (4) 避免数字或指针的下标越界, 特别要当心发生“多 1”或者“少 1”操作
- (5) 动态内存的申请与释放必须配对, 防止内存泄漏
- (6) 用 `free` 或 `delete` 释放了内存之后, 立即将指针设置为 `NULL`, 防止“野指针”
- (7) 使用智能指针。

3. 内存泄露及解决办法:

什么是内存泄露?

简单地说就是申请了一块内存空间, 使用完毕后没有释放掉。(1) `new` 和 `malloc` 申请资源使用后, 没有用 `delete` 和 `free` 释放; (2) 子类继承父类时, 父类析构函数不是虚函数。(3) Windows 句柄资源使用后没有释放。

怎么检测?

第一: 良好的编码习惯, 使用了内存分配的函数, 一旦使用完毕, 要记得使用其相应的函数释放掉。

第二: 将分配的内存的指针以链表的形式自行管理, 使用完毕之后从链表中删除, 程序结束时可检查改链表。

第三: 使用智能指针。

第四: 一些常见的工具插件, 如 `ccmalloc`、`Dmalloc`、`Leaky`、`Valgrind` 等等。

19、`malloc` 和局部变量分配在堆还是栈?

`malloc` 是在堆上分配内存, 需要程序员自己回收内存; 局部变量是在栈中分配内存, 超过作用域就自动回收。

20、简述 C++ 中内存对齐的使用场景

内存对齐应用于三种数据类型中: struct/class/union

struct/class/union 内存对齐原则有四个:

1.数据成员对齐规则: 结构(struct)或联合(union)的数据成员, 第一个数据成员放在 offset 为 0 的地方, 以后每个数据成员存储的起始位置要从该成员大小或者成员的子成员大小的整数倍开始。

2.结构体作为成员:如果一个结构里有某些结构体成员,则结构体成员要从其内部"最宽基本类型成员"的整数倍地址开始存储。(struct a 里存有 struct b,b 里有 char,int ,double 等元素,那 b 应该从 8 的整数倍开始存储)。

3.收尾工作:结构体的总大小,也就是 sizeof 的结果,必须是其内部最大成员的"最宽基本类型成员"的整数倍。不足的要补齐。(基本类型不包括 struct/class/union)。

4.sizeof(union), 以结构里面 size 最大元素为 union 的 size, 因为在某一时刻, union 只有一个成员真正存储于该地址。

21、简述一下什么是面向对象

1.面向对象是一种编程思想, 把一切东西看成是一个个对象, 比如人、耳机、鼠标、水杯等, 他们各自都有属性, 比如: 耳机是白色的, 鼠标是黑色的, 水杯是圆柱形的等等, 把这些对象拥有的属性变量和操作这些属性变量的函数打包成一个类来表示

2.面向过程和面向对象的区别

面向过程: 根据业务逻辑从上到下写代码

面向对象: 将数据与函数绑定到一起, 进行封装, 这样能够更快速的开发程序, 减少了重复代码的重写过程

22、简述一下面向对象的三大特征

面向对象的三大特征是封装、继承、多态。

封装: 将数据和操作数据的方法进行有机结合, 隐藏对象的属性和实现细节, 仅对外公开接口来和对象进行交互。封装本质上是一种管理: 我们如何管理兵马俑呢? 比如如果什么都不管, 兵马俑就被随意破坏了。那么我们首先建了一座房子把兵马俑给封装起来。但是我们目

的全封装起来, 不让别人看。所以我们开放了售票通道, 可以买票突破封装在合理的监管机制下进去参观。类也是一样, 不想给别人看到的, 我们使用 `protected/private` 把成员封装起来。开放一些共有的成员函数对成员合理的访问。所以封装本质是一种管理。

继承: 可以使用现有类的所有功能, 并在无需重新编写原来的类的情况下对这些功能进行扩展。

三种继承方式

继承方式 `private` 继承 `protected` 继承 `public` 继承

基类的 `private` 成员 不可见 不可见 不可见

基类的 `protected` 成员 变为 `private` 成员 仍为 `protected` 成员 仍为 `protected` 成员

基类的 `public` 成员 变为 `private` 成员 变为 `protected` 成员 仍为 `public` 成员 仍为 `public` 成员

多态: 用父类型别的指针指向其子类的实例, 然后通过父类的指针调用实际子类的成员函数。实现多态, 有二种方式, 重写, 重载。

23、C++中虚函数与纯虚函数的区别

虚函数和纯虚函数可以定义在同一个类中, 含有纯虚函数的类被称为抽象类, 而只含有虚函数的类不能被称为抽象类。

虚函数可以被直接使用, 也可以被子类重载以后, 以多态的形式调用, 而纯虚函数必须在子类中实现该函数才可以使用, 因为纯虚函数在基类有声明而没有定义。

虚函数和纯虚函数都可以在子类中被重载, 以多态的形式被调用。

虚函数和纯虚函数通常存在于抽象基类之中, 被继承的子类重载, 目的是提供一个统一的接口。

虚函数的定义形式: `virtual{};` 纯虚函数的定义形式: `virtual {} = 0;` 在虚函数和纯虚函数的定义中不能有 `static` 标识符, 原因很简单, 被 `static` 修饰的函数在编译时要求前期绑定, 然而虚函数却是动态绑定, 而且被两者修饰的函数生命周期也不一样。

24、STL 的基本组成部分

标准模板库 (Standard Template Library, 简称 STL) 简单说, 就是一些常用数据结构和算法的模板的集合。

广义上讲, STL 分为 3 类: Algorithm (算法)、Container (容器) 和 Iterator (迭代器),

容器和算法通过迭代器可以进行无缝地连接。

详细的说, STL 由 6 部分组成: 容器(Container)、算法 (Algorithm)、 迭代器 (Iterator)、仿函数 (Function object)、适配器 (Adaptor)、空间配制器 (Allocator)。

25、简述 vector 的实现原理

vector 底层实现原理为一维数组 (元素在空间连续存放)。

26、vector 和 list 的区别, 分别适用于什么场景?

vector 和 list 区别在于底层实现机理不同, 因而特性和适用场景也有所不同。

vector: 一维数组

特点: 元素在内存连续存放, 动态数组, 在堆中分配内存, 元素连续存放, 有保留内存, 如果减少大小后内存也不会释放。

优点: 和数组类似开辟一段连续的空间, 并且支持随机访问, 所以它的查找效率高其时间复杂度 $O(1)$ 。

缺点: 由于开辟一段连续的空间, 所以插入删除会需要对数据进行移动比较麻烦, 时间复杂度 $O(n)$, 另外当空间不足时还需要进行扩容。

list: 双向链表

特点: 元素在堆中存放, 每个元素都是存放在一块内存中, 它的内存空间可以是不连续的, 通过指针来进行数据的访问。

优点: 底层实现是循环双链表, 当对大量数据进行插入删除时, 其时间复杂度 $O(1)$ 。

缺点: 底层没有连续的空间, 只能通过指针来访问, 所以查找数据需要遍历其时间复杂度 $O(n)$, 没有提供[]操作符的重载。

应用场景

vector 拥有一段连续的内存空间, 因此支持随机访问, 如果需要高效的随即访问, 而不在乎插入和删除的效率, 使用 vector。

list 拥有一段不连续的内存空间, 如果需要高效的插入和删除, 而不关心随机访问, 则应使用 list。

3.3.2 Java 语言

1、为什么 Java 代码可以实现一次编写、到处运行？

JVM（Java 虚拟机）是 Java 跨平台的关键。

在程序运行前，Java 源代码（.java）需要经过编译器编译成字节码（.class）。在程序运行时，JVM 负责将字节码翻译成特定平台下的机器码并运行，也就是说，只要在不同的平台上安装对应的 JVM，就可以运行字节码文件。

同一份 Java 源代码在不同的平台上运行，它不需要做任何的改变，并且只需要编译一次。而编译好的字节码，是通过 JVM 这个中间的“桥梁”实现跨平台的，JVM 是与平台相关的软件，它能将统一的字节码翻译成该平台的机器码。

注意事项

编译的结果是生成字节码、不是机器码，字节码不能直接运行，必须通过 JVM 翻译成机器码才能运行；

跨平台的是 Java 程序、而不是 JVM，JVM 是用 C/C++开发的软件，不同平台下需要安装不同版本的 JVM。

2、一个 Java 文件里可以有多个类吗（不含内部类）？

一个 java 文件里可以有多个类，但最多只能有一个被 public 修饰的类；

如果这个 java 文件中包含 public 修饰的类，则这个类的名称必须和 java 文件名一致。

3、说一说你对 Java 访问权限的了解

Java 语言为我们提供了三种访问修饰符，即 private、protected、public，在使用这些修饰符修饰目标时，一共可以形成四种访问权限，即 private、default、protected、public，注意在不加任何修饰符时为 default 访问权限。

在修饰成员变量/成员方法时，该成员的四访问权限的含义如下：

private：该成员可以被该类内部成员访问；

default：该成员可以被该类内部成员访问，也可以被同一包下其他的类访问；

protected：该成员可以被该类内部成员访问，也可以被同一包下其他的类访问，还可以被它的子类访问；

public: 该成员可以被任意包下, 任意类的成员进行访问。

在修饰类时, 该类只有两种访问权限, 对应的访问权限的含义如下:

default: 该类可以被同一包下其他的类访问;

public: 该类可以被任意包下, 任意的类所访问。

4、介绍全局变量和局部变量的区别

Java 中的变量分为成员变量和局部变量, 它们的区别如下:

成员变量:

1.成员变量是在类的范围里定义的变量;

2.成员变量有默认初始值;

3.未被 **static** 修饰的成员变量也叫实例变量, 它存储于对象所在的堆内存中, 生命周期与对象相同;

4.被 **static** 修饰的成员变量也叫类变量, 它存储于方法区中, 生命周期与当前类相同。

局部变量:

1.局部变量是在方法里定义的变量;

2.局部变量没有默认初始值;

3.局部变量存储于栈内存中, 作用的范围结束, 变量空间会自动的释放。

注意事项

Java 中没有真正的全局变量, 面试官应该是出于其他语言的习惯说全局变量的, 他的本意应该是指成员变量。

5、如何对 Integer 和 Double 类型判断相等?

Integer、Double 不能直接进行比较, 这包括:

不能用 **==** 进行直接比较, 因为它们是不同的数据类型;

不能转为字符串进行比较, 因为转为字符串后, 浮点值带小数点, 整数值不带, 这样它们永远都不相等;

不能使用 **compareTo** 方法进行比较, 虽然它们都有 **compareTo** 方法, 但该方法只能对相同类型进行比较。

整数、浮点类型的包装类, 都继承于 **Number** 类型, 而 **Number** 类型分别定义了将数字转

换为 `byte`、`short`、`int`、`long`、`float`、`double` 的方法。所以，可以将 `Integer`、`Double` 先转为转换为相同的基本数据类型（如 `double`），然后使用`==`进行比较。

6、面向对象的三大特征是什么？

面向对象的程序设计方法具有三个基本特征：封装、继承、多态。其中，封装指的是将对象的实现细节隐藏起来，然后通过一些公用方法来暴露该对象的功能；继承是面向对象实现软件复用的重要手段，当子类继承父类后，子类作为一种特殊的父类，将直接获得父类的属性和方法；多态指的是子类对象可以直接赋给父类变量，但运行时依然表现出子类的行为特征，这意味着同一个类型的对象在执行同一个方法时，可能表现出多种行为特征。

7、封装的目的是什么，为什么要有封装？

封装是面向对象编程语言对客观世界的模拟，在客观世界里，对象的状态信息都被隐藏在对象内部，外界无法直接操作和修改。对一个类或对象实现良好的封装，可以实现以下目的：

隐藏类的实现细节；

让使用者只能通过事先预定的方法来访问数据，从而可以在该方法里加入控制逻辑，限制对成员变量的不合理访问；

可进行数据检查，从而有利于保证对象信息的完整性；

便于修改，提高代码的可维护性。

8、说一说你对多态的理解

因为子类其实是一种特殊的父类，因此 `Java` 允许把一个子类对象直接赋给一个父类引用变量，无须任何类型转换，或者被称为向上转型，向上转型由系统自动完成。

当把一个子类对象直接赋给父类引用变量时，例如 `BaseClass obj = new SubClass();`，这个 `obj` 引用变量的编译时类型是 `BaseClass`，而运行时类型是 `SubClass`，当运行时调用该引用变量的方法时，其方法行为总是表现出子类方法的行为特征，而不是父类方法的行为特征，这就可能出现：相同类型的变量、调用同一个方法时呈现出多种不同的行为特征，这就是多态。

9、String 可以被继承吗？

`String` 类由 `final` 修饰，所以不能被继承。

10、String 和 StringBuffer 有什么区别

String 类是不可变类，即一旦一个 String 对象被创建以后，包含在这个对象中的字符序列是不可改变的，直至这个对象被销毁。

StringBuffer 对象则代表一个字符序列可变的字符串，当一个 StringBuffer 被创建以后，通过 StringBuffer 提供的 append()、insert()、reverse()、setCharAt()、setLength()等方法可以改变这个字符串对象的字符序列。一旦通过 StringBuffer 生成了最终想要的字符串，就可以调用它的 toString()方法将其转换为一个 String 对象。

11、接口和抽象类有什么区别？

从设计目的上来说，二者有如下的区别：

接口体现的是一种规范。对于接口的实现者而言，接口规定了实现者必须向外提供哪些服务；对于接口的调用者而言，接口规定了调用者可以调用哪些服务，以及如何调用这些服务。当在一个程序中使用接口时，接口是多个模块间的耦合标准；当在多个应用程序之间使用接口时，接口是多个程序之间的通信标准。

抽象类体现的是一种模板式设计。抽象类作为多个子类的抽象父类，可以被当成系统实现过程中的中间产品，这个中间产品已经实现了系统的部分功能，但这个产品依然不能当成最终产品，必须有更进一步的完善，这种完善可能有几种不同方式。

从使用方式上来说，二者有如下的区别：

接口里只能包含抽象方法、静态方法、默认方法和私有方法，不能为普通方法提供方法实现；抽象类则完全可以包含普通方法。

接口里只能定义静态常量，不能定义普通成员变量；抽象类里则既可以定义普通成员变量，也可以定义静态常量。

接口里不包含构造器；抽象类里可以包含构造器，抽象类里的构造器并不是用于创建对象，而是让其子类调用这些构造器来完成属于抽象类的初始化操作。

接口里不能包含初始化块；但抽象类则完全可以包含初始化块。

一个类最多只能有一个直接父类，包括抽象类；但一个类可以直接实现多个接口，通过实现多个接口可以弥补 Java 单继承的不足。

12、谈谈你对面向接口编程的理解

接口体现的是一种规范和实现分离的设计哲学，充分利用接口可以极好地降低程序各模块之间的耦合，从而提高系统的可扩展性和可维护性。基于这种原则，很多软件架构设计理论都倡导“面向接口”编程，而不是面向实现类编程，希望通过面向接口编程来降低程序的耦合。

13、遇到过异常吗，如何处理？

在 Java 中，可以按照如下三个步骤处理异常：

1.捕获异常

将业务代码包裹在 `try` 块内部，当业务代码中发生任何异常时，系统都会为此异常创建一个异常对象。创建异常对象之后，JVM 会在 `try` 块之后寻找可以处理它的 `catch` 块，并将异常对象交给这个 `catch` 块处理。

2.处理异常

在 `catch` 块中处理异常时，应该先记录日志，便于以后追溯这个异常。然后根据异常的类型、结合当前的业务情况，进行相应的处理。比如，给变量赋予一个默认值、直接返回空值、向外抛出一个新的业务异常交给调用者处理，等等。

3.回收资源

如果业务代码打开了某个资源，比如数据库连接、网络连接、磁盘文件等，则需要在这段业务代码执行完毕后关闭这项资源。并且，无论是否发生异常，都要尝试关闭这项资源。将关闭资源的代码写在 `finally` 块内，可以满足这种需求，即无论是否发生异常，`finally` 块内的代码总会被执行。

14、说一说 Java 的异常机制

关于异常处理：

在 Java 中，处理异常的语句由 `try`、`catch`、`finally` 三部分组成。其中，`try` 块用于包裹业务代码，`catch` 块用于捕获并处理某个类型的异常，`finally` 块则用于回收资源。当业务代码发生异常时，系统会创建一个异常对象，然后由 JVM 寻找可以处理这个异常的 `catch` 块，并将异常对象交给这个 `catch` 块处理。若业务代码打开了某项资源，则可以在 `finally` 块中关闭这项资源，因为无论是否发生异常，`finally` 块一定会执行。

关于抛出异常：

当程序出现错误时，系统会自动抛出异常。除此以外，Java 也允许程序主动抛出异常。当业务代码中，判断某项错误的条件成立时，可以使用 `throw` 关键字向外抛出异常。在这种情况下，如果当前方法不知道该如何处理这个异常，可以在方法签名上通过 `throws` 关键字声明抛出异常，则该异常将交给 JVM 处理。

关于异常跟踪栈：

程序运行时，经常会发生一系列方法调用，从而形成方法调用栈。异常机制会导致异常在这些方法之间传播，而异常传播的顺序与方法的调用相反。异常从发生异常的方法向外传播，首先传给该方法的调用者，再传给上层调用者，以此类推。最终会传到 `main` 方法，若依然没有得到处理，则 JVM 会终止程序，并打印异常跟踪栈的信息

15、finally 是无条件执行的吗？

不管 `try` 块中的代码是否出现异常，也不管哪一个 `catch` 块被执行，甚至在 `try` 块或 `catch` 块中执行了 `return` 语句，`finally` 块总会被执行。

16、说一说你对泛型的理解

Java 集合有个缺点一把一个对象“丢进”集合里之后，集合就会“忘记”这个对象的数据类型，当再次取出该对象时，该对象的编译类型就变成了 `Object` 类型（其运行时类型没变）。

Java 集合之所以被设计成这样，是因为集合的设计者不知道我们会用集合来保存什么类型的对象，所以他们把集合设计成能保存任何类型的对象，只要求具有很好的通用性。但这样做带来如下两个问题：

集合对元素类型没有任何限制，这样可能引发一些问题。例如，想创建一个只能保存 `Dog` 对象的集合，但程序也可以轻易地将 `Cat` 对象“丢”进去，所以可能引发异常。

由于把对象“丢进”集合时，集合丢失了对象的状态信息，只知道它盛装的是 `Object`，因此取出集合元素后通常还需要进行强制类型转换。这种强制类型转换既增加了编程的复杂度，也可能引发 `ClassCastException` 异常。

从 Java 5 开始，Java 引入了“参数化类型”的概念，允许程序在创建集合时指定集合元素的类型，Java 的参数化类型被称为泛型（`Generic`）。例如 `List<String>`，表明该 `List` 只能保存字符串类型的对象。

有了泛型以后，程序再也不能“不小心”地把其他对象“丢进”集合中。而且程序更加简

洁，集合自动记住所有集合元素的数据类型，从而无须对集合元素进行强制类型转换。

17、说一说你对 Java 反射机制的理解

Java 程序中的对象在运行时可以表现为两种类型，即编译时类型和运行时类型。例如 `Person p = new Student();`，这行代码将会生成一个 `p` 变量，该变量的编译时类型为 `Person`，运行时类型为 `Student`。

有时，程序在运行时接收到外部传入的一个对象，该对象的编译时类型是 `Object`，但程序又需要调用该对象的运行时类型的方法。这就要求程序需要在运行时发现对象和类的真实信息，而解决这个问题有以下两种做法：

第一种做法是假设在编译时和运行时都完全知道类型的具体信息，在这种情况下，可以先使用 `instanceof` 运算符进行判断，再利用强制类型转换将其转换成其运行时类型的变量即可。

第二种做法是编译时根本无法预知该对象和类可能属于哪些类，程序只依靠运行时信息来发现该对象和类的真实信息，这就必须使用反射。

具体来说，通过反射机制，我们可以实现如下的操作：

程序运行时，可以通过反射获得任意一个类的 `Class` 对象，并通过这个对象查看这个类的信息；

程序运行时，可以通过反射创建任意一个类的实例，并访问该实例的成员；

程序运行时，可以通过反射机制生成一个类的动态代理类或动态代理对象。

18、TreeMap 的底层原理

`TreeMap` 基于红黑树（`Red-Black tree`）实现。映射根据其键的自然顺序进行排序，或者根据创建映射时提供的 `Comparator` 进行排序，具体取决于使用的构造方法。`TreeMap` 的基本操作 `containsKey`、`get`、`put`、`remove` 方法，它的时间复杂度是 $\log(N)$ 。

`TreeMap` 包含几个重要的成员变量：`root`、`size`、`comparator`。其中 `root` 是红黑树的根节点。它是 `Entry` 类型，`Entry` 是红黑树的节点，它包含了红黑树的 6 个基本组成：`key`、`value`、`left`、`right`、`parent` 和 `color`。`Entry` 节点根据根据 `Key` 排序，包含的内容是 `value`。`Entry` 中 `key` 比较大小是根据比较器 `comparator` 来进行判断的。`size` 是红黑树的节点个数。

19、Map 和 Set 有什么区别？

Set 代表无序的，元素不可重复的集合；

Map 代表具有映射关系（key-value）的集合，其所有的 key 是一个 Set 集合，即 key 无序且不能重复。

20、介绍一下 Java 的序列化与反序列化

序列化机制可以将对象转换成字节序列，这些字节序列可以保存在磁盘上，也可以在网络中传输，并允许程序将这些字节序列再次恢复成原来的对象。其中，对象的序列化（Serialize），是指将一个 Java 对象写入 IO 流中，对象的反序列化（Deserialize），则是指从 IO 流中恢复该 Java 对象。

若对象要支持序列化机制，则它的类需要实现 Serializable 接口，该接口是一个标记接口，它没有提供任何方法，只是标明该类是可以序列化的，Java 的很多类已经实现了 Serializable 接口，如包装类、String、Date 等。

若要实现序列化，则需要使用对象流 ObjectOutputStream 和 ObjectInputStream。其中，在序列化时需要调用 ObjectOutputStream 对象的 writeObject() 方法，以输出对象序列。在反序列化时需要调用 ObjectInputStream 对象的 readObject() 方法，将对象序列恢复为对象。

21、创建线程有哪几种方式？

创建线程有三种方式，分别是继承 Thread 类、实现 Runnable 接口、实现 Callable 接口。

通过继承 Thread 类来创建并启动线程的步骤如下：

1. 定义 Thread 类的子类，并重写该类的 run() 方法，该 run() 方法将作为线程执行体。
2. 创建 Thread 子类的实例，即创建了线程对象。
3. 调用线程对象的 start() 方法来启动该线程。

通过实现 Runnable 接口来创建并启动线程的步骤如下：

1. 定义 Runnable 接口的实现类，并实现该接口的 run() 方法，该 run() 方法将作为线程执行体。
2. 创建 Runnable 实现类的实例，并将其作为 Thread 的 target 来创建 Thread 对象，Thread 对象为线程对象。
3. 调用线程对象的 start() 方法来启动该线程。

通过实现 Callable 接口来创建并启动线程的步骤如下：

1. 创建 `Callable` 接口的实现类, 并实现 `call()` 方法, 该 `call()` 方法将作为线程执行体, 且该 `call()` 方法有返回值。然后再创建 `Callable` 实现类的实例。
2. 使用 `FutureTask` 类来包装 `Callable` 对象, 该 `FutureTask` 对象封装了该 `Callable` 对象的 `call()` 方法的返回值。
3. 使用 `FutureTask` 对象作为 `Thread` 对象的 `target` 创建并启动新线程。
4. 调用 `FutureTask` 对象的 `get()` 方法来获得子线程执行结束后的返回值。

22、线程是否可以重复启动, 会有什么后果?

只能对处于新建状态的线程调用 `start()` 方法, 否则将引发 `IllegalThreadStateException` 异常。

23、如何实现线程同步?

1. 同步方法

即有 `synchronized` 关键字修饰的方法, 由于 `java` 的每个对象都有一个内置锁, 当用此关键字修饰方法时, 内置锁会保护整个方法。在调用该方法前, 需要获得内置锁, 否则就处于阻塞状态。需要注意, `synchronized` 关键字也可以修饰静态方法, 此时如果调用该静态方法, 将会锁住整个类。

2. 同步代码块

即有 `synchronized` 关键字修饰的语句块, 被该关键字修饰的语句块会自动被加上内置锁, 从而实现同步。需值得注意的是, 同步是一种高开销的操作, 因此应该尽量减少同步的内容。通常没有必要同步整个方法, 使用 `synchronized` 代码块同步关键代码即可。

3. ReentrantLock

`Java 5` 新增了一个 `java.util.concurrent` 包来支持同步, 其中 `ReentrantLock` 类是可重入、互斥、实现了 `Lock` 接口的锁, 它与使用 `synchronized` 方法和快具有相同的基本行为和语义, 并且扩展了其能力。需要注意的是, `ReentrantLock` 还有一个可以创建公平锁的构造方法, 但由于能大幅度降低程序运行效率, 因此不推荐使用。

4. volatile

`volatile` 关键字为域变量的访问提供了一种免锁机制, 使用 `volatile` 修饰域相当于告诉虚拟机该域可能会被其他线程更新, 因此每次使用该域就要重新计算, 而不是使用寄存器中的值。需要注意的是, `volatile` 不会提供任何原子操作, 它也不能用来修饰 `final` 类型的变量。

5.原子变量

在 java 的 `util.concurrent.atomic` 包中提供了创建了原子类型变量的工具类, 使用该类可以简化线程同步。例如 `AtomicInteger` 表可以用原子方式更新 `int` 的值, 可用在应用程序中 (如以原子方式增加的计数器), 但不能用于替换 `Integer`。可扩展 `Number`, 允许那些处理机遇数字类的工具和实用工具进行统一访问。

24、Java 多线程之间的通信方式

在 Java 中线程通信主要有以下三种方式:

1.wait()、notify()、notifyAll()

如果线程之间采用 `synchronized` 来保证线程安全, 则可以利用 `wait()`、`notify()`、`notifyAll()` 来实现线程通信。这三个方法都不是 `Thread` 类中所声明的方法, 而是 `Object` 类中声明的方法。原因是每个对象都拥有锁, 所以让当前线程等待某个对象的锁, 当然应该通过这个对象来操作。并且因为当前线程可能会等待多个线程的锁, 如果通过线程来操作, 就非常复杂了。另外, 这三个方法都是本地方法, 并且被 `final` 修饰, 无法被重写。

`wait()`方法可以让当前线程释放对象锁并进入阻塞状态。`notify()`方法用于唤醒一个正在等待相应对象锁的线程, 使其进入就绪队列, 以便在当前线程释放锁后竞争锁, 进而得到 CPU 的执行。`notifyAll()`用于唤醒所有正在等待相应对象锁的线程, 使它们进入就绪队列, 以便在当前线程释放锁后竞争锁, 进而得到 CPU 的执行。

每个锁对象都有两个队列, 一个是就绪队列, 一个是阻塞队列。就绪队列存储了已就绪 (将要竞争锁) 的线程, 阻塞队列存储了被阻塞的线程。当一个阻塞线程被唤醒后, 才会进入就绪队列, 进而等待 CPU 的调度。反之, 当一个线程被 `wait` 后, 就会进入阻塞队列, 等待被唤醒。

2.await()、signal()、signalAll()

如果线程之间采用 `Lock` 来保证线程安全, 则可以利用 `await()`、`signal()`、`signalAll()`来实现线程通信。这三个方法都是 `Condition` 接口中的方法, 该接口是在 Java 1.5 中出现的, 它用来替代传统的 `wait+notify` 实现线程间的协作, 它的使用依赖于 `Lock`。相比使用 `wait+notify`, 使用 `Condition` 的 `await+signal` 这种方式能够更加安全和高效地实现线程间协作。

`Condition` 依赖于 `Lock` 接口, 生成一个 `Condition` 的基本代码是 `lock.newCondition()`。必须要注意的是, `Condition` 的 `await()/signal()/signalAll()` 使用都必须在 `lock` 保护之内, 也就是说, 必须在 `lock.lock()` 和 `lock.unlock` 之间才可以使用。事实上, `await()/signal()/signalAll()` 与

wait()/notify()/notifyAll()有着天然的对对应关系。即：Condition 中的 await()对应 Object 的 wait(), Condition 中的 signal()对应 Object 的 notify(), Condition 中的 signalAll()对应 Object 的 notifyAll()。

3.BlockingQueue

Java 5 提供了一个 BlockingQueue 接口, 虽然 BlockingQueue 也是 Queue 的子接口, 但它的主要用途并不是作为容器, 而是作为线程通信的工具。BlockingQueue 具有一个特征: 当生产者线程试图向 BlockingQueue 中放入元素时, 如果该队列已满, 则该线程被阻塞; 当消费者线程试图从 BlockingQueue 中取出元素时, 如果该队列已空, 则该线程被阻塞。

程序的两个线程通过交替向 BlockingQueue 中放入元素、取出元素, 即可很好地控制线程的通信。线程之间需要通信, 最经典的场景就是生产者与消费者模型, 而 BlockingQueue 就是针对该模型提供的解决方案。

25、sleep() 和 wait() 的区别

sleep()是 Thread 类中的静态方法, 而 wait()是 Object 类中的成员方法;
sleep()可以在任何地方使用, 而 wait()只能在同步方法或同步代码块中使用;
sleep()不会释放锁, 而 wait()会释放锁, 并需要通过 notify()/notifyAll()重新获取锁。

26、阻塞线程的方式有哪些?

当发生如下情况时, 线程将会进入阻塞状态:

线程调用 sleep()方法主动放弃所占用的处理器资源;

线程调用了一个阻塞式 IO 方法, 在该方法返回之前, 该线程被阻塞;

线程试图获得一个同步监视器, 但该同步监视器正被其他线程所持有;

线程在等待某个通知 (notify) ;

程序调用了线程的 suspend()方法将该线程挂起, 但这个方法容易导致死锁, 所以应该尽量避免使用该方法。

27、JVM 什么时候触发 GC, 如何减少 Full GC 的次数?

当 Eden 区的空间耗尽时 Java 虚拟机便会触发一次 Minor GC 来收集新生代的垃圾, 存活下来的对象, 则会被送到 Survivor 区, 简单说就是当新生代的 Eden 区满的时候触发 Minor GC。

serial GC 中, 老年代内存剩余已经小于之前年轻代晋升老年代的平均大小, 则进行 Full GC。而在 CMS 等并发收集器中则是每隔一段时间检查一下老年代内存的使用量, 超过一定比例时进行 Full GC 回收。

可以采用以下措施来减少 Full GC 的次数:

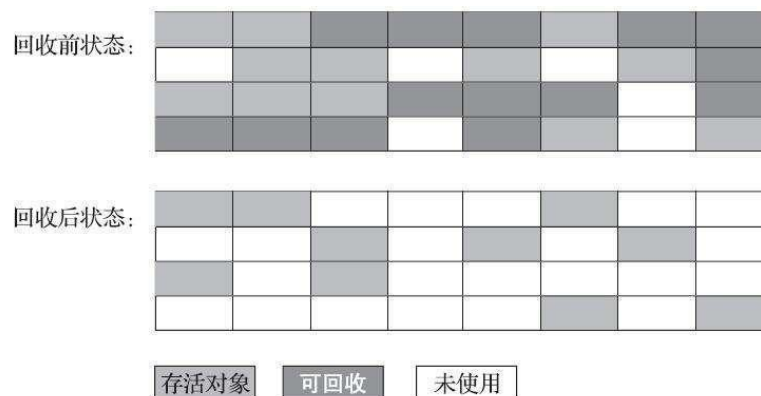
- 增加方法区的空间;
- 增加老年代的空间;
- 减少新生代的空间;
- 禁止使用 `System.gc()` 方法;
- 使用标记-整理算法, 尽量保持较大的连续内存空间;
- 排查代码中无用的大对象。

28、说一说你对 GC 算法的了解。

标记-清除算法:

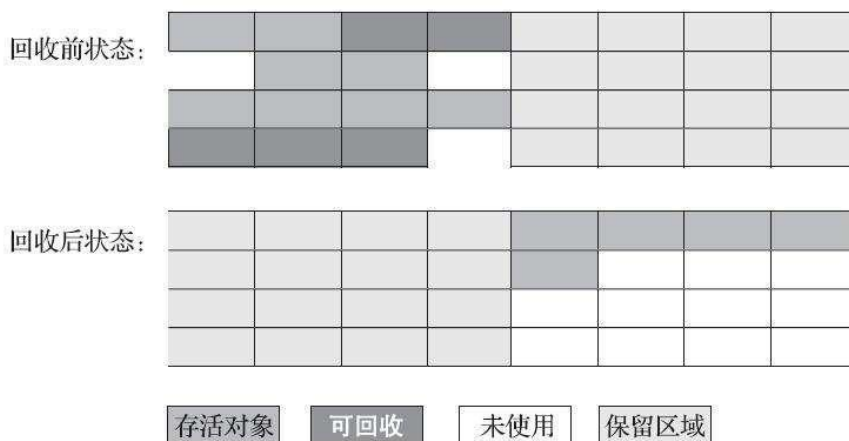
最早出现也是最基础的垃圾收集算法是“标记-清除”(Mark-Sweep)算法, 如它的名字一样, 算法分为“标记”和“清除”两个阶段: 首先标记出所有需要回收的对象, 在标记完成后, 统一回收掉所有被标记的对象, 也可以反过来, 标记存活的对象, 统一回收所有未被标记的对象。

它的主要缺点有两个: 第一个是执行效率不稳定, 如果 Java 堆中包含大量对象, 而且其中大部分是需要被回收的, 这时必须进行大量标记和清除的动作, 导致标记和清除两个过程的执行效率都随对象数量增长而降低; 第二个是内存空间的碎片化问题, 标记、清除之后会产生大量不连续的内存碎片, 空间碎片太多可能会导致当以后在程序运行过程中需要分配较大对象时无法找到足够的连续内存而不得不提前触发另一次垃圾收集动作。标记-清除算法的执行过程如下图所示。



标记-复制算法:

为了解决标记-清除算法面对大量可回收对象时执行效率低的问题, 1969 年 Fenichel 提出了一种称为“半区复制”(Semispace Copying)的垃圾收集算法, 它将可用内存按容量划分为大小相等的两块, 每次只使用其中的一块。当这一块的内存用完了, 就将还存活着的对象复制到另外一块上面, 然后再把已使用过的内存空间一次清理掉。如果内存中多数对象都是存活的, 这种算法将会产生大量的内存间复制的开销, 但对于多数对象都是可回收的情况, 算法需要复制的就是占少数的存活对象, 而且每次都是针对整个半区进行内存回收, 分配内存时也就不用考虑有空间碎片的复杂情况, 只要移动堆顶指针, 按顺序分配即可。这样实现简单, 运行高效, 不过其缺陷也显而易见, 这种复制回收算法的代价是将可用内存缩小为了原来的一半, 空间浪费未免太多了一点。标记-复制算法的执行过程如下图所示。



在 1989 年, Andrew Appel 针对具备“朝生夕灭”特点的对象, 提出了一种更优化的半区复制分代策略, 现在称为“Appel 式回收”。Appel 式回收的具体做法是把新生代分为一块较大的 Eden 空间和两块较小的 Survivor 空间, 每次分配内存只使用 Eden 和其中一块 Survivor。发生垃圾搜集时, 将 Eden 和 Survivor 中仍然存活的对象一次性复制到另外一块 Survivor 空间上, 然后直接清理掉 Eden 和已用过的那块 Survivor 空间。HotSpot 虚拟机默认 Eden 和 Survivor 的大小比例是 8 : 1, 也即每次新生代中可用内存空间为整个新生代容量的 90% (Eden 的 80% 加上一个 Survivor 的 10%), 只有一个 Survivor 空间, 即 10% 的新生代是会被“浪费”的。当然, 98% 的对象可被回收仅仅是“普通场景”下测得的数据, 任何人都没有办法百分百保证每次回收都只有不多于 10% 的对象存活, 因此 Appel 式回收还有一个充当罕见情况的“逃生门”的安全设计, 当 Survivor 空间不足以容纳一次 Minor GC 之后存活的对象时, 就需要依赖其他内存区域 (实际上大多就是老年代) 进行分配担保 (Handle Promotion)。

标记-整理算法:

标记-复制算法在对象存活率较高时就要进行较多的复制操作,效率将会降低。更关键的是,如果不想浪费 50%的空间,就需要有额外的空间进行分配担保,以应对被使用的内存中所有对象都 100%存活的极端情况,所以在老年代一般不能直接选用这种算法。

针对老年代对象的存亡特征,1974 年 Edward Lueders 提出了另外一种有针对性的“标记-整理”(Mark-Compact)算法,其中的标记过程仍然与“标记-清除”算法一样,但后续步骤不是直接对可回收对象进行清理,而是让所有存活的对象都向内存空间一端移动,然后直接清理掉边界以外的内存,“标记-整理”算法的示意图如下图所示。



noobdream.com

3.3.3 数据结构

1、红黑树和 AVL 树的定义，特点，以及二者区别

平衡二叉树（AVL 树）：

平衡二叉树又称为 AVL 树，是一种特殊的二叉排序树。其左右子树都是平衡二叉树，且左右子树高度之差的绝对值不超过 1。一句话表述为：以树中所有结点为根的树的左右子树高度之差的绝对值不超过 1。将二叉树上结点的左子树深度减去右子树深度的值称为平衡因子 BF，那么平衡二叉树上的所有结点的平衡因子只可能是 -1、0 和 1。只要二叉树上有一个结点的平衡因子的绝对值大于 1，则该二叉树就是不平衡的。

红黑树：

红黑树是一种二叉查找树，但在每个节点增加一个存储位表示节点的颜色，可以是红或黑（非红即黑）。通过对任何一条从根到叶子的路径上各个节点着色的方式的限制，红黑树确保没有一条路径会比其它路径长出两倍，因此，红黑树是一种弱平衡二叉树，相对于要求严格的 AVL 树来说，它的旋转次数少，所以对于搜索，插入，删除操作较多的情况下，通常使用红黑树。

性质：

1. 每个节点非红即黑
2. 根节点是黑的;
3. 每个叶节点（叶节点即树尾端 NULL 指针或 NULL 节点）都是黑的;
4. 如果一个节点是红色的，则它的子节点必须是黑色的。
5. 对于任意节点而言，其到叶子点树 NULL 指针的每条路径都包含相同数目的黑节点;

区别：

AVL 树是高度平衡的，频繁的插入和删除，会引起频繁的 rebalance，导致效率下降；红黑树不是高度平衡的，算是一种折中，插入最多两次旋转，删除最多三次旋转。

2、哈夫曼编码

哈夫曼编码是哈夫曼树的一种应用，广泛用于数据文件压缩。哈夫曼编码算法用字符在文件中出现的频率来建立使用 0, 1 表示个字符的最优表示方式，其具体算法如下：

(1)哈夫曼算法以自底向上的方式构造表示最优前缀码的二叉树 T。

(2)算法以 $|C|$ 个叶结点开始, 执行 $|C|-1$ 次的“合并”运算后产生最终所要求的树 T 。

(3)假设编码字符集中每一字符 c 的频率是 $f(c)$ 。以 f 为键值的优先队列 Q 用在贪心选择时有效地确定算法当前要合并的2棵具有最小频率的树。一旦2棵具有最小频率的树合并后, 产生一棵新的树, 其频率为合并的2棵树的频率之和, 并将新树插入优先队列 Q 。经过 $n-1$ 次的合并后, 优先队列中只剩下一棵树, 即所要求的树 T 。

3、map 底层为什么用红黑树实现

1、红黑树:

红黑树是一种二叉查找树, 但在每个节点增加一个存储位表示节点的颜色, 可以是红或黑 (非红即黑)。通过对任何一条从根到叶子的路径上各个节点着色的方式的限制, 红黑树确保没有一条路径会比其它路径长出两倍, 因此, 红黑树是一种弱平衡二叉树, 相对于要求严格的AVL树来说, 它的旋转次数少, 所以对于搜索, 插入, 删除操作较多的情况下, 通常使用红黑树。

性质:

1. 每个节点非红即黑
2. 根节点是黑的;
3. 每个叶节点 (叶节点即树尾端 NULL 指针或 NULL 节点) 都是黑的;
4. 如果一个节点是红色的, 则它的子节点必须是黑色的。
5. 对于任意节点而言, 其到叶子点树 NULL 指针的每条路径都包含相同数目的黑节点;

2、平衡二叉树 (AVL 树):

红黑树是在 AVL 树的基础上提出来的。

平衡二叉树又称为 AVL 树, 是一种特殊的二叉排序树。其左右子树都是平衡二叉树, 且左右子树高度之差的绝对值不超过 1。

AVL 树中所有结点为根的树的左右子树高度之差的绝对值不超过 1。

将二叉树上结点的左子树深度减去右子树深度的值称为平衡因子 BF, 那么平衡二叉树上的所有结点的平衡因子只可能是-1、0 和 1。只要二叉树上有一个结点的平衡因子的绝对值大于 1, 则该二叉树就是不平衡的。

3、红黑树较 AVL 树的优点:

AVL 树是高度平衡的, 频繁的插入和删除, 会引起频繁的 rebalance, 导致效率下降; 红黑

树不是高度平衡的，算是一种折中，插入最多两次旋转，删除最多三次旋转。

所以红黑树在查找，插入删除的性能都是 $O(\log n)$ ，且性能稳定，所以 STL 里面很多结构包括 map 底层实现都是使用的红黑树。

4、介绍一下 B+树

B+是一种多路搜索树，主要为磁盘或其他直接存取辅助设备而设计的一种平衡查找树，在 B+树中，每个节点的可以有多个孩子，并且按照关键字大小有序排列。所有记录节点都是按照键值的大小顺序存放在同一层的叶节点中。相比 B 树，其具有以下几个特点：

每个节点上的指针上限为 $2d$ 而不是 $2d+1$ (d 为节点的出度)

内节点不存储 data,只存储 key

叶子节点不存储指针

5、map 和 unordered_map 的底层实现

map 底层是基于红黑树实现的，因此 map 内部元素排列是有序的。而 unordered_map 底层则是基于哈希表实现的，因此其元素的排列顺序是杂乱无序的。

6、哈希表的桶个数为什么是质数，合数有何不妥？

哈希表的桶个数使用质数，可以最大程度减少冲突概率，使哈希后的数据分布的更加均匀。如果使用合数，可能会造成很多数据分布会集中在某些点上，从而影响哈希表效率。

7、解决 hash 冲突的方法

当哈希表关键字集合很大时，关键字值不同的元素可能会映射到哈希表的同一地址上，这样的现象称为哈希冲突。目前常用的解决哈希冲突的方法如下：

开放定址法：当发生地址冲突时，按照某种方法继续探测哈希表中的其他存储单元，直到找到空位置为止。

再哈希法：当发生哈希冲突时使用另一个哈希函数计算地址值，直到冲突不再发生。这种方法不易产生聚集，但是增加计算时间，同时需要准备许多哈希函数。

链地址法：将所有哈希值相同的 Key 通过链表存储。key 按顺序插入到链表中

建立公共溢出区：采用一个溢出表存储产生冲突的关键字。如果公共溢出区还产生冲突，

再采用处理冲突方法处理。

8、深度优先搜索遍历和广度优先搜索遍历的过程

深度优先搜索遍历

基本思想：首先访问出发点 v ，并将其标记为已访问；然后选取与 v 邻接的未被访问的邻接顶点 w ，访问 w ；再选取与 w 邻接的未被访问的顶点访问，以此类推。当一个顶点所有的邻接顶点都被访问过时，则依次退回最近被访问过的顶点，若该顶点还有其他邻接顶点未被访问，则从这些顶点中去一个顶点进行上述的过程，直至图中所有顶点都被访问过为止。

广度优先搜索遍历

基本思想：首先访问起始顶点 v ，然后选取与 v 邻接的全部顶点 $w_1, w_2, \dots, w_n$ 进行访问，再一次访问与 $w_1, w_2, \dots, w_n$ 邻接的全部顶点（不包括已访问过的顶点），以此类推，直至所有顶点都被访问过为止。

9、迪杰斯特拉算法的过程

该算法可以求得某一顶点到其余各顶点的最短路径。

算法思想：设有两个顶点集合 S 和 T ，其中集合 S 中存放的是图中已找到最短路径的顶点，集合 T 中存放的是图中的剩余顶点。

初始状态时，集合 S 中只包含源点 v_0 ，然后不断从集合 T 中选取到顶点 v_0 路径最短的顶点 v_u 并加入集合 S 中。集合 S 每加入一个新的顶点 v_u ，都要修改 v_0 到集合 T 中各个顶点的最短路径的长度值。不断重复这个过程，直至集合 T 中的顶点全部并入到 S 中为止。

10、图的存储方式有哪些

- ① 邻接矩阵：是图的顺序存储结构，用两个数组分别存储数据元素（顶点）信息和数据元素之间的关系（边/弧）的信息。图的邻接矩阵表示是唯一的，无向图的邻接矩阵是对称的。
- ② 邻接表：是图的链式存储结构，由单链表的表头形成的顶点表和单链表其余结点所形成的边表两部分组成。
- ③ 十字链表：有向图的另一种链式存储结构。
- ④ 邻接多重表：无向图的链式存储结构。

11、图的深度遍历是否唯一？

不一定是唯一。我们可以取图中任一顶点进行深度遍历。

12、如何实现循环队列？

如何实现：把数组弄成一个环，让 **rear** 和 **front** 指针沿着环走，这样就可以产生循环队列。

好处：循环队列是顺序队列的改进，在顺序队列中，在元素进队的时候，**rear** 要向后移动，元素出队的时候，**front** 也要向后移动，这样经过一系列的出队和入队操作之后，两个指针最后会达到数组的末端，此时虽然队中已经没有元素了，但是还是不能让元素入队，即出现了“假溢出”的现象。循环队列就能避免出现这个现象。

13、单向链表如何判断链表中是否出现了环

单链表有环，是指单链表中某个节点的 **next** 指针域指向的是链表中在它之前的某一个节点，这样在链表的尾部形成一个环形结构。

解法：定义两个指针，同时从链表的头节点出发，一个指针一次走一步，另一个指针一次走两步。如果走得快的指针追上了走得慢的指针，那么链表就是环形链表；如果走得快的指针走到了链表的末尾（**next** 指向 **NULL**）都没有追上第一个指针，那么链表就不是环形链表。

noobdream.com

3.3.4 操作系统

1、线程和进程的区别？

- (1) 一个线程从属于一个进程；一个进程可以包含多个线程。
- (2) 一个线程挂掉，对应的进程挂掉；一个进程挂掉，不会影响其他进程。
- (3) 进程是系统资源调度的最小单位；线程 CPU 调度的最小单位。
- (4) 进程系统开销显著大于线程开销；线程需要的系统资源更少。
- (5) 进程在执行时拥有独立的内存单元，多个线程共享进程的内存，如代码段、数据段、扩展段；但每个线程拥有自己的栈段和寄存器组。
- (6) 进程切换时需要刷新 TLB 并获取新的地址空间，然后切换硬件上下文和内核栈，线程切换时只需要切换硬件上下文和内核栈。
- (7) 通信方式不一样。
- (8) 进程适应于多核、多机分布；线程适用于多核

2、什么是协程？

协程：协程是微线程，在子程序内部执行，可在子程序内部中断，转而执行别的子程序，在适当的时候再返回来接着执行。

3、为什么协程比线程切换的开销小？

- (1) 协程执行效率极高。协程直接操作栈基本没有内核切换的开销，所以上下文的切换非常快，切换开销比线程更小。
- (2) 协程不需要多线程的锁机制，因为多个协程从属于一个线程，不存在同时写变量冲突，效率比线程高。避免了加锁解锁的开销。

4、进程切换为什么比线程更消耗资源？

进程切换时需要刷新 TLB 并获取新的地址空间，然后切换硬件上下文和内核栈；线程切换时只需要切换硬件上下文和内核栈。

5、介绍一下进程之间的通信。

为了提高计算机系统的效率, 增强计算机系统内各种硬件的并行操作能力, 操作系统要求程序结构必须适应并发处理的需要, 为此引入了进程的概念。而进程并行时, 需要考虑进程间的通信, 进程间通信主要有以下几种方式: 匿名管道、命名管道、信号、消息队列、共享内存、信号量、Socket。

6、介绍一下信号量。

在多进程环境下, 为了防止多个进程同时访问一个公共资源而出现问题, 需要一种方法来协调各个进程, 保证它们能够合理地使用公共资源。信号量就是这样一种机制。

信号量的数据类型为结构 `sem_t`, 它本质上是一个长整型的数。函数 `sem_init()` 用来初始化一个信号量。它的原型为:

```
extern int sem_init_P ((sem_t *_sem, int _pshared, unsigned int _value));
```

`sem` 为指向信号量结构的一个指针; `pshared` 不为 0 时此信号量在进程间共享, 否则只能为当前进程的所有线程共享; `value` 给出了信号量的初始值。

(1) 函数 `sem_post(sem_t *sem)` 用来增加信号量的值。当有线程阻塞在这个信号量上时, 调用这个函数会使其中的一个线程不在阻塞, 选择机制同样是由线程的调度策略决定的。

(2) 函数 `sem_wait(sem_t *sem)` 被用来阻塞当前线程直到信号量 `sem` 的值大于 0, 解除阻塞后将 `sem` 的值减一, 表明公共资源经使用后减少。函数 `sem_trywait(sem_t *sem)` 是函数 `sem_wait()` 的非阻塞版本, 它直接将信号量 `sem` 的值减一。

(3) 函数 `sem_timedwait(sem_t *sem, const struct timespec *abs_timeout)` 与 `sem_wait()` 类似, 只不过 `abs_timeout` 指定一个阻塞的时间上限, 如果调用因不能立即执行递减而要阻塞。

(4) 函数 `sem_destroy(sem_t *sem)` 用来释放信号量 `sem`。

7、僵尸进程和孤儿进程。

我们知道在 `unix/linux` 中, 正常情况下, 子进程是通过父进程创建的, 子进程在创建新的进程。子进程的结束和父进程的运行是一个异步过程, 即父进程永远无法预测子进程到底什么时候结束。当一个进程完成它的工作终止之后, 它的父进程需要调用 `wait()` 或者 `waitpid()` 系统调用取得子进程的终止状态。

孤儿进程: 一个父进程退出, 而它的一个或多个子进程还在运行, 那么那些子进程将成为孤儿进程。孤儿进程将被 `init` 进程(进程号为 1)所收养, 并由 `init` 进程对它们完成状态收集工作。

僵尸进程：一个进程使用 `fork` 创建子进程，如果子进程退出，而父进程并没有调用 `wait` 或 `waitpid` 获取子进程的状态信息，那么子进程的进程描述符仍然保存在系统中。这种进程称之为僵尸进程。

8、介绍线程之间的通信方式。

锁机制：包括互斥锁、条件变量、读写锁互斥锁提供了以排他方式防止数据结构被并发修改的方法。读写锁允许多个线程同时读共享数据，而对写操作是互斥的。条件变量可以以原子的方式阻塞进程，直到某个特定条件为真为止。对条件的测试是在互斥锁的保护下进行的。条件变量始终与互斥锁一起使用。

信号量机制(Semaphore)：包括无名线程信号量和命名线程信号量

信号机制(Signal)：类似进程间的信号处理线程间的通信目的主要是用于线程同步，所以线程没有像进程通信中的用于数据交换的通信机制。

9、说一说进程的状态。

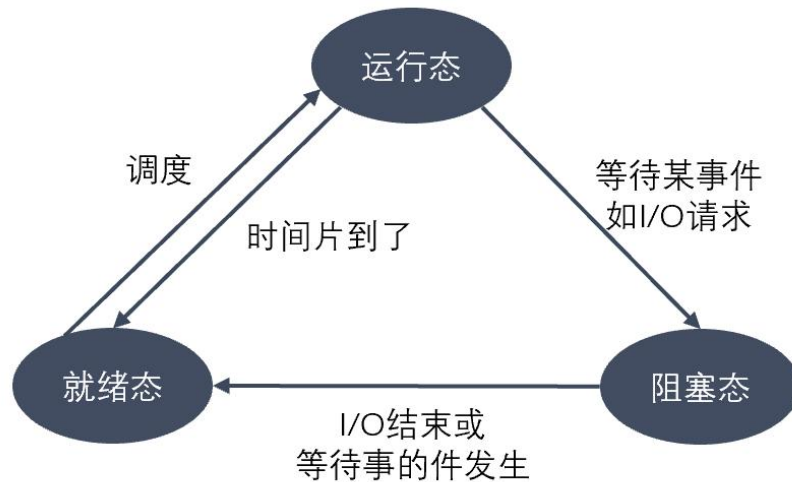
进程的 3 种基本状态：运行、就绪和阻塞。

(1) 就绪：当一个进程获得了除处理机以外的一切所需资源，一旦得到处理机即可运行，则称此进程处于就绪状态。就绪进程可以按多个优先级来划分队列。例如，当一个进程由于时间片用完而进入就绪状态时，排入低优先级队列；当进程由 I/O 操作完成而进入就绪状态时，排入高优先级队列。

(2) 运行：当一个进程在处理机上运行时，则称该进程处于运行状态。处于此状态的进程的数目小于等于处理器的数目，对于单处理机系统，处于运行状态的进程只有一个。在没有其他进程可以执行时（如所有进程都在阻塞状态），通常会执行系统的空闲进程。

(3) 阻塞：也称为等待或睡眠状态，一个进程正在等待某一事件发生（例如请求 I/O 而等待 I/O 完成等）而暂时停止运行，这时即使把处理机分配给进程也无法运行，故称该进程处于阻塞状态。

其转移图如下：



进程的五种状态

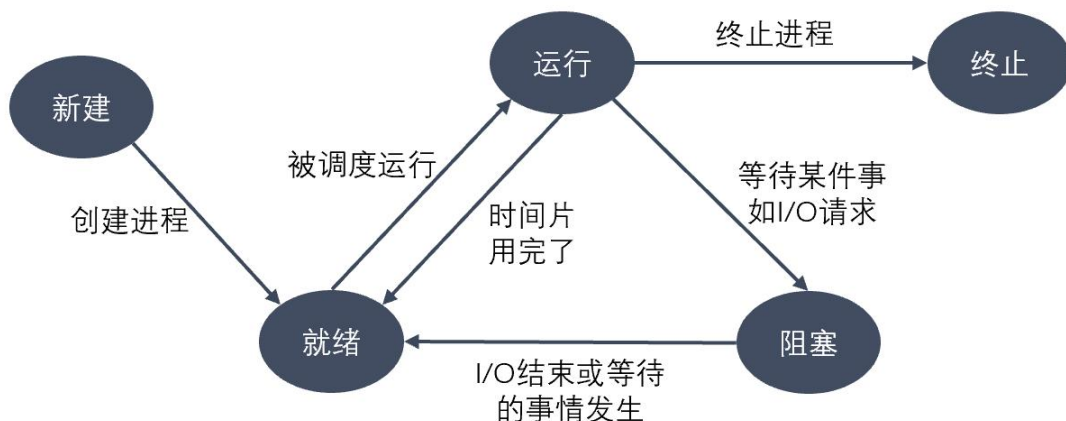
创建状态: 进程在创建时需要申请一个空白 **PCB**, 向其中填写控制和管理进程的信息, 完成资源分配。如果创建工作无法完成, 比如资源无法满足, 就无法被调度运行, 把此时进程所处状态称为创建状态

就绪状态: 进程已经准备好, 已分配到所需资源, 只要分配到 **CPU** 就能够立即运行

执行状态: 进程处于就绪状态被调度后, 进程进入执行状态

阻塞状态: 正在执行的进程由于某些事件 (**I/O** 请求, 申请缓存区失败) 而暂时无法运行, 进程受到阻塞。在满足请求时进入就绪状态等待系统调用

终止状态: 进程结束, 或出现错误, 或被系统终止, 进入终止状态。无法再执行



10、CPU 调度的最小单位是什么？线程需要 CPU 调度吗？

进程是 CPU 分配资源的最小单位，线程是 CPU 调度的最小单位。

线程是比进程更小的能独立运行的基本单位，需要通过 CPU 调度来切换上下文，达到并发的目的。

11、进程之间共享内存的通信方式有什么好处？

采用共享内存通信的一个显而易见的好处是效率高，因为进程可以直接读写内存，而不需要任何数据的拷贝。对于像管道和消息队列等通信方式，则需要在内核和用户空间进行四次的数据拷贝，而共享内存则只拷贝两次数据：一次从输入文件到共享内存区，另一次从共享内存区到输出文件。

实际上，进程之间在共享内存时，并不总是读写少量数据后就解除映射，有新的通信时，再重新建立共享内存区域。而是保持共享区域，直到通信完毕为止，这样，数据内容一直保存在共享内存中，并没有写回文件。共享内存中的内容往往是在解除映射时才写回文件的。因此，采用共享内存的通信方式效率是非常高的。

12、什么情况下会产生死锁？

如果在计算机系统中同时具备下面四个必要条件时，那么会发生死锁。换句话说，只要下面四个条件有一个不具备，系统就不会出现死锁。

1.互斥条件。即某个资源在一段时间内只能由一个进程占有，不能同时被两个或两个以上的进程占有。这种独占资源如 CD-ROM 驱动器，打印机等等，必须在占有该资源的进程主动释放它之后，其它进程才能占有该资源。这是由资源本身的属性所决定的。如独木桥就是一种独占资源，两方的人不能同时过桥。

2.不剥夺条件。进程所获得的资源在未使用完毕之前，资源申请者不能强行地从资源占有者手中夺取资源，而只能由该资源的占有者进程自行释放。如过独木桥的人不能强迫对方后退，也不能非法地将对方推下桥，必须是桥上的人自己过桥后空出桥面（即主动释放占有资源），对方的人才能过桥。

3.请求和保持条件。进程至少已经占有一个资源，但又申请新的资源；由于该资源已被另外进程占有，此时该进程阻塞；但是，它在等待新资源之时，仍继续占用已占有的资源。还以过独木桥为例，甲乙两人在桥上相遇。甲走过一段桥面（即占有了一些资源），还需要走其余

的桥面（申请新的资源），但那部分桥面被乙占有（乙走过一段桥面）。甲过不去，前进不能，又不后退；乙也处于同样的状况。

4.循环等待条件。存在一个进程等待序列 $\{P_1, P_2, \dots, P_n\}$ ，其中 P_1 等待 P_2 所占有的某一资源， P_2 等待 P_3 所占有的某一源，.....，而 P_n 等待 P_1 所占有的的某一资源，形成一个进程循环等待环。就像前面的过独木桥问题，甲等待乙占有的桥面，而乙又等待甲占有的桥面，从而彼此循环等待。

上面提到的这四个条件在死锁时会同时发生。也就是说，只要有一个必要条件不满足，则死锁就可以排除。

13、说一说你对自旋锁的理解。

旋锁的定义：当一个线程尝试去获取某一把锁的时候，如果这个锁此时已经被别人获取(占用)，那么此线程就无法获取到这把锁，该线程将会等待，间隔一段时间后会再次尝试获取。这种采用循环加锁 -> 等待的机制被称为自旋锁(spinlock)。

自旋锁有以下特点

用于临界区互斥

在任何时刻最多只能有一个执行单元获得锁

要求持有锁的处理器所占用的时间尽可能短

等待锁的线程进入忙循环

自旋锁存在的问题

如果某个线程持有锁的时间过长，就会导致其它等待获取锁的线程进入循环等待，消耗CPU。使用不当会造成CPU使用率极高。

无法满足等待时间最长的线程优先获取锁。不公平的锁就会存在“线程饥饿”问题。

自旋锁的优点

自旋锁不会使线程状态发生切换，一直处于用户态，即线程一直都是 active 的；不会使线程进入阻塞状态，减少了不必要的上下文切换，执行速度快

非自旋锁在获取不到锁的时候会进入阻塞状态，从而进入内核态，当获取到锁的时候需要从内核态恢复，需要线程上下文切换。（线程被阻塞后便进入内核（Linux）调度状态，这个会导致系统在用户态与内核态之间来回切换，严重影响锁的性能）

自旋锁与互斥锁的区别

自旋锁与互斥锁都是为了实现保护资源共享的机制。

无论是自旋锁还是互斥锁，在任意时刻，都最多只能有一个保持者。

获取互斥锁的线程，如果锁已经被占用，则该线程将进入睡眠状态；获取自旋锁的线程则不会睡眠，而是一直循环等待锁释放。

14、说一说你对悲观锁的理解。

悲观锁总是假设最坏的情况，每次去拿数据的时候都认为别人会修改，所以每次在拿数据的时候都会上锁，这样别人想拿这个数据就会阻塞直到它拿到锁（共享资源每次只给一个线程使用，其它线程阻塞，用完后再把资源转让给其它线程）。传统的关系型数据库里边就用到了很多这种锁机制，比如行锁，表锁等，读锁，写锁等，都是在做操作之前先上锁。

15、说一说你对乐观锁的理解。

乐观锁总是假设最好的情况，每次去拿数据的时候都认为别人不会修改，所以不会上锁，但是在更新的时候会判断一下在此期间别人有没有去更新这个数据，可以使用版本号机制和 CAS 算法实现。乐观锁适用于多读的应用类型，这样可以提高吞吐量，像数据库提供的类似于 write_condition 机制，其实都是提供的乐观锁。

16、select 和 epoll 的区别。

select 函数监视的文件描述符分 3 类，分别是 writefds、readfds、和 exceptfds。调用后 select 函数会阻塞，直到有描述符就绪（有数据 可读、可写、或者有 except），或者超时（timeout 指定等待时间，如果立即返回设为 null 即可），函数返回。当 select 函数返回后，可以通过遍历 fdset，来找到就绪的描述符。

select 目前几乎在所有的平台上支持，其良好跨平台支持也是它的一个优点。select 的一个缺点在于单个进程能够监视的文件描述符的数量存在最大限制，在 Linux 上一般为 1024，可以通过修改宏定义甚至重新编译内核的方式提升这一限制，但是这样也会造成效率的降低。内核需要传递消息到用户空间，需要内存拷贝

相对于 select 和 poll 来说，epoll 更加灵活，没有描述符限制。epoll 使用一个文件描述符管理多个描述符，将用户关系的文件描述符的事件存放到内核的一个事件表中，这样在用户空间和内核空间的 copy 只需一次。

epoll 能打开的 FD 的上限远大于 1024（1G 的内存上能监听约 10 万个端口）。

效率提升，不是轮询的方式，不会随着 FD 数目的增加效率下降。只有活跃可用的 FD 才会调用 callback 函数；即 Epoll 最大的优点就在于它只管你“活跃”的连接，而跟连接总数无关，因此在实际的网络环境中，Epoll 的效率就会远远高于 select 和 poll。

17、epoll 有哪两种模式？

epoll 对文件描述符的操作有两种模式：LT（level trigger）和 ET（edge trigger）。LT 模式是默认模式，LT 模式与 ET 模式的区别如下：

LT 模式：当 epoll_wait 检测到描述符事件发生并将此事件通知应用程序，应用程序可以不立即处理该事件。下次调用 epoll_wait 时，会再次响应应用程序并通知此事件。

ET 模式：当 epoll_wait 检测到描述符事件发生并将此事件通知应用程序，应用程序必须立即处理该事件。如果不处理，下次调用 epoll_wait 时，不会再次响应应用程序并通知此事件。

LT 模式

LT(level triggered)是缺省的工作方式，并且同时支持 block 和 no-block socket。在这种做法中，内核告诉你一个文件描述符是否就绪了，然后你可以对这个就绪的 fd 进行 IO 操作。如果你不作任何操作，内核还是会继续通知你的。

ET 模式

ET(edge-triggered)是高速工作方式，只支持 no-block socket。在这种模式下，当描述符从未就绪变为就绪时，内核通过 epoll 告诉你。然后它会假设你知道文件描述符已经就绪，并且不会再为那个文件描述符发送更多的就绪通知，直到你做了某些操作导致那个文件描述符不再为就绪状态了(比如，你在发送，接收或者接收请求，或者发送接收的数据少于一定量时导致了一个 EWOULDBLOCK 错误)。但是请注意，如果一直不对这个 fd 作 IO 操作(从而导致它再次变成未就绪)，内核不会发送更多的通知(only once)。

ET 模式在很大程度上减少了 epoll 事件被重复触发的次数，因此效率要比 LT 模式高。epoll 工作在 ET 模式的时候，必须使用非阻塞套接口，以避免由于一个文件句柄的阻塞读/阻塞写操作把处理多个文件描述符的任务饿死。

18、说一说你对 Linux 内核态与用户态的了解。

内核态其实从本质上说就是内核，它是一种特殊的软件程序，控制计算机的硬件资源，例

如协调 CPU 资源，分配内存资源，并且提供稳定的环境供应用程序运行。

用户态就是提供应用程序运行的空间，为了使应用程序访问到内核管理的资源例如 CPU，内存，I/O。内核必须提供一组通用的访问接口，这些接口就叫系统调用。

系统调用是操作系统的的功能单位。根据不同的应用场景，不同的 Linux 发行版本提供的系统调用数量也不尽相同，大致在 240-350 之间。这些系统调用组成了用户态跟内核态交互的基本接口。

从用户态到内核态切换可以通过三种方式：

系统调用：系统调用本身就是中断，但是是软件中断，跟硬中断不同。

异常：如果当前进程运行在用户态，如果这个时候发生了异常事件，就会触发切换。例如：缺页异常。

外设中断：当外设完成用户的请求时，会向 CPU 发送中断信号。

19、虚拟内存

物理内存是有限的（即使支持了热插拔）、非连续的，不同的 CPU 架构对物理内存的组织都不同。这使得直接使用物理内存非常复杂，为了降低使用内存的复杂度，引入了虚拟内存机制。

虚拟内存抽象了应用程序物理内存的细节，只允许物理内存保存所需的信息（按需分页），并提供了一种保护和控制进程间数据共享数据的机制。有了虚拟内存机制之后，每次访问可以使用更易理解的虚拟地址，让 CPU 转换成实际的物理地址访问内存，降低了直接使用、管理物理内存的门槛。

物理内存按大小被分成页框、页，每块物理内存可以被映射为一个或多个虚拟内存页。这块映射关系，由操作系统的页表来保存，页表是有层级的。层级最低的页表，保存实际页面的物理地址，较高层级的页表包含指向低层级页表的物理地址，指向顶级的页表的地址，驻留在寄存器中。当执行地址转换时，先从寄存器获取顶级页表地址，然后依次索引，找到具体页面的物理地址。

20、page cache

从外部存储介质中加载数据到内存中，这个过程是比较耗时的，因为外部存储介质读写性能毫秒级。为了减少外部存储设备的读写，Linux 内核提供了 Page cache。最常见的操作，每

次读取文件时，数据都会被放入页面缓存中，以避免后续读取时所进行昂贵的磁盘访问。同样，当写入文件时，数据被重新放置在缓存中，被标记为脏页，定期的更新到存储设备上，以提高读写性能。

21、OOM killer

机器上的内存可能会被耗尽，并且内核将无法回收足够的内存用于运行新的程序，为了保存系统的其余部分，内核会调用 OOM killer 杀掉一些进程，以释放内存。

段页机制

段页机制是操作系统管理内存的一种方式，简单的来说，就是如何管理、组织系统中的内存。要理解这种机制，需要了解一下内存寻址的发展历程。

直接寻址：早期的内存很小，通过硬编码的形式，直接定位到内存地址。这种方式有着明显的缺点：可控性弱、难以重定位、难以维护

分段机制：8086 处理器，寻址空间达到 1MB，即地址线扩展了 20 位，由于制作 20 位的寄存器较为困难，为了能在 16 位的寄存器的基础上，寻址 20 位的地址空间，引入了段的概念，即内存地址=段基址左移 4 位+偏移

分页机制：随着寻址空间的进一步扩大、虚拟内存技术的引入，操作系统引入了分页机制。引入分页机制后，逻辑地址经过段机制转换得到的地址仅是中间地址，还需要通过页机制转换，才能得到实际的物理地址。逻辑地址 -->(分段机制) 线性地址 -->(分页机制) 物理地址。

22、谈谈虚拟内存模型。

虚拟内存分成五大区，分别为栈区、堆区、全局区（静态区）、文字常量区（常量存储区）、程序代码区。五大区特性如下：

栈区（stack）：由编译器自动分配释放，存放函数的参数值，局部变量的值等。其操作方式类似于数据结构中的栈。

堆区（heap）：一般由程序员分配释放，若程序员不释放，程序结束时可能由 OS 回收。注意它与数据结构中的堆是两回事，分配方式倒是类似于链表。

全局区（静态区）（static）：全局变量和静态变量的存储是放在一块的，初始化的全局变量和静态变量在一块区域，未初始化的全局变量和未初始化的静态变量在相邻的另一块区域。程序结束后由系统释放。

文字常量区（常量存储区）：常量字符串就是放在这里的。程序结束后由系统释放。这是一块比较特殊的存储区，他们里面存放的是常量，不允许修改。

程序代码区：存放函数体的二进制代码。

23、什么是物理内存和虚拟内存，为什么要有虚拟内存？

物理内存及虚拟内存定义

物理内存是相对于虚拟内存而言的。物理内存指通过物理内存条而获得的内存空间，而虚拟内存则是指将硬盘的一块区域划分来作为内存。内存主要作用是在计算机运行时为操作系统和各种程序提供临时储存。

为什么要有虚拟内存

在早期的计算机中，要运行一个程序，会把这些程序全都装入内存，程序都是直接运行在内存上的，也就是说程序中访问的内存地址都是实际的物理内存地址。当计算机同时运行多个程序时，必须保证这些程序用到的内存总量要小于计算机实际物理内存的大小。

24、内存和缓存有什么区别？

内存和缓存是计算机不同的组成部件。

内存特性

内存也被称作内存存储器，其作用是用于暂时存放 CPU 的运算数据，以及与硬盘等外部存储交换的数据。只要计算机在运行中，CPU 就会把需要进行运算的数据调到内存中进行运算，当运算完成后 CPU 再将结果传送出来，内存的运行也决定了计算机的稳定运行。

缓存特性

CPU 芯片面积和成本的因素影响，决定了缓存都很小。现在一般的缓存不过几 M，CPU 缓存的运行频率极高，一般是和处理器同频运作，工作效率远远大于系统内存和硬盘。实际工作时，CPU 往往需要重复读取读取同样的数据块，而缓存容量的增大，可以大幅度提升 CPU 内部读取数据的命中率，而不用再到内存或者硬盘上寻找，以此提高系统性能。

25、深拷贝和浅拷贝的区别是什么，它们各自的使用场景是什么？

浅拷贝只是对指针的拷贝，拷贝后两个指针指向同一个内存空间；深拷贝不断对指针进行拷贝，而且对指针指向的内容进行拷贝，经深拷贝后的指针是指向两个不同的地址空间。

浅拷贝

对一个已知对象进行拷贝时，编译系统会自动调用一次构造函数（拷贝构造函数），如果用户未定义拷贝构造函数，则会调用默认拷贝构造函数，调用一次构造函数，调用两次析构函数，两个对象的指针成员所指内存相同，但是程序结束时该内存被释放了两次，会造成内存泄漏问题。

深拷贝

在对含有指针成员的对象进行拷贝时，必须要自己定义拷贝构造函数，使拷贝后的对象指针成员有自己的内存空间，即进行深拷贝，这样就避免了内存泄漏的发生，调用一次构造函数，一次自定义拷贝构造函数，两次析构函数。两个对象的指针成员所指内容不同。

26、五大 I/O 模型

Linux 有五大 I/O 模型，分别为阻塞 IO、同步非阻塞 IO、IO 多路复用、信号驱动 IO、异步 IO。

26、什么是 IO

我们都知道 unix 世界里，一切皆文件。而文件是什么呢？文件就是一串二进制流而已。无论是 socket，还是 FIFO、管道、终端，对我们来说，一切都是文件，一切都是流。在信息交换的过程中，我们都是对这些流进行数据的收发操作简称为 I/O 操作(input and output)。往流中读出数据，系统调用 read；写入数据，系统调用 write。

计算机里有这么多的流，我怎么知道要操作哪个流呢？

做到这个的就是文件描述符，即通常所说的 fd，一个 fd 就是一个整数，所以对这个整数的操作就是对这个文件（流）的操作。我们创建一个 socket，通过系统调用会返回一个文件描述符，那么剩下对 socket 的操作就会转化为对这个描述符的操作。不能不说这又是一种分层和抽象的思想。

27、IO 交互

对于一个输入操作来说，进程 IO 系统调用后，内核会先看缓冲区有没有相应的缓存数据，没有的话再到设备中读取，因为设备 IO 一般速度较慢，需要等待，内核缓冲区有数据则直接复制到进程空间。所以，对于一个网络输入操作通常包括两个不同阶段：

(1) 等待网络数据到达网卡->读取到内核缓冲区

(2) 从内核缓冲区复制数据->用户空间

IO 有内存 IO、网络 IO 和磁盘 IO 三种, 通常我们所说的 IO 指的是网络 IO 和磁盘 IO 两者。

28、说说缺页中断机制。

缺页中断

在请求分页系统中, 可以通过查询页表中的状态位来确定所要访问的页面是否存在于内存中。当所要访问的页面不在内存时, 会产生一次缺页中断, 此时操作系统会根据页表中的外存地址在外存中找到所缺的一页, 将其调入内存。 缺页中断的处理流程如下:

- A. 在内存中有空闲物理页面时, 分配一物理页帧 f , 转第 E 步;
- B. 依据页面置换算法选择被替换的物理页帧 f , 对应逻辑页 q ;
- C. 如果 q 被修改过, 则把它写回外存;
- D. 修改 q 的页表项中驻留位置为 0;
- E. 将需要访问的页 p 装入到物理页 f ;
- F. 修改 p 的页表项驻留位为 1, 物理页帧号为 f ;
- G. 重新执行产生缺页的指令。

缺页中断与一般的中断存在区别

(1) 范围不同

一般中断只需要保护现场然后就直接跳到需及时处理的地方;

缺页中断除了保护现场之外, 还要判断内存中是否有足够的空间存储所需的页或段, 然后再把所需页调进来再使用。

(2) 结果不同

一般中断在处理完之后返回时, 执行下一条指令;

缺页中断返回时, 执行产生中断的那一条指令。

(3) 次数不同

一般中断只产生一次, 发生中断指令后转入相应处理程序进行处理, 恢复被中断程序现场; 在指令执行期间产生和处理缺页中断信号, 一条指令在执行期间, 可能产生多次缺页中断。

29、软中断和硬中断有什么区别？

硬中断

由与系统相连的外设(比如网卡、硬盘)自动产生的。主要是用来通知操作系统外设状态的变化。比如当网卡收到数据包的时候, 就会发出一个中断。我们通常所说的中断指的是硬中断(hardirq)。

软中断

为了满足实时系统的要求, 中断处理应该是越快越好。Linux 为了实现这个特点, 当中断发生的时候, 硬中断处理那些短时间就可以完成的工作, 而将那些处理事件比较长的工作, 放到中断之后来完成, 也就是软中断(softirq)来完成。

中断嵌套

Linux 下硬中断是可以嵌套的, 但是没有优先级的概念, 也就是说任何一个新的中断都可以打断正在执行的中断, 但同种中断除外。软中断不能嵌套, 但相同类型的软中断可以在不同CPU 上并行执行。

软中断与硬中断之间的区别

- (1) 硬中断是由外部事件引起的因此具有随机性和突发性; 软中断是执行中断指令产生的, 无外面事件中断请求信号, 因此软中断的发生不是随机的而是由程序安排好的;
- (2) 硬中断的中断号是由中断控制器提供的; 软中断的中断号是由指令直接给出的, 无需使用中断控制器。
- (3) 硬中断的中断响应周期, CPU 需要发中断回合信号; 软中断的中断响应周期, CPU 不需要发中断回合信号。
- (4) 硬中断是可屏蔽的; 软中断是不可屏蔽的。

30、介绍一下你对 CopyOnWrite 的了解。

CopyOnWrite (写时拷贝技术)

Linux 的 fork()使用写时拷贝页来实现新进程的创建, 它是一种可推迟甚至避免数据拷贝的技术, 开始时内核并不会复制整个地址空间, 而是让父子进程共享地址空间, 只有在写时才复制地址空间, 使得父子进程都拥有独立的地址空间, 即资源的复制是在只有需要写入时才会发生。在此之前都是以读的方式去和父进程共享资源, 这样, 在页根本不会被写入的场景下, fork()立即执行 exec(), 无需对地址空间进行复制, fork()的实际开销就是复制父进程的一个页表和为

子进程创建一个进程描述符, 也就是说只有当进程空间中各段的内存内容发生变化时, 父进程才将其内容复制一份传给子进程, 大大提高了效率。

写时拷贝技术优缺点

写时拷贝技术是一种很重要的优化手段, 核心是懒惰处理实体资源请求, 在多个实体资源之间只是共享资源, 起初是并不真正实现资源拷贝, 只有当实体有需要对资源进行修改时才真正为实体分配私有资源。但写时拷贝技术也有它的优点和缺点:

优点: 写时拷贝技术可以减少分配和复制大量资源时带来的瞬间延时, 但实际上是将这种延时附加到了后续的操作之中。

缺点: 写时拷贝技术可以减少不必要的资源分配。比如 `fork` 进程时, 并不是所有的页面都需要复制, 父进程的代码段和只读数据段都不被允许修改, 所以无需复制。

31、页面置换算法有哪些? 什么是 LRU?

① 最佳置换算法 (OPT): 在预知一个进程的页面号引用串的情况下, 每次都淘汰以后不再使用的页以后

最迟再被使用的页面。该算法不能实现, 只能作为一个标准来衡量其他置换算法的优劣。

② 先进先出算法 (FIFO): 每次总是淘汰最先进入内存的页面, 也就是将在内存中驻留时间最长的页面

淘汰。(可能会产生 Belady 异常, 缺页次数随着分配的物理块的增加而增加)。

③ 最近最少使用算法 (LRU): 选择最近最少未被使用的页面淘汰, 其思想是用以前的页面引用情况来预

测将来会出现的页面引用情况。利用了局部性原理。

④ 时钟置换算法 (CLOCK): 是 LRU 和 FIFO 的折中, 具体方法略。

⑤ 工作集算法

⑥ 工作集时钟算法

⑦ 第二次机会算法

⑧ 最近未使用 (NRU)

32、进程和程序的区别

① 进程是动态的, 程序是静止的。进程是程序的执行, 程序是有序代码的集合。

- ② 进程是暂时的，程序是永久的。进程是一个状态变化的过程，程序可以长久保存。
- ③ 进程和程序的组成不同：进程包括程序，数据和进程控制块。
- ④ 进程和程序是密切相关的。通过多次执行，一个程序可以对应多个进程；通过调度关系，一个进程可以包括多个程序。
- ⑤ 进程可以创建其他进程，但是程序不能形成新的程序。

33、操作系统中的磁盘调度算法

磁盘调度算法目的：使磁盘的平均寻道时间最少。

- 1) 先来先服务 (FCFS) 算法：它按照输入输出请求到达的顺序，逐一完成访问请求，它只考虑请求访问者的先后次序，而不考虑它们要访问的物理位置。
- 2) 最短查找时间优先 (SSTF) 算法：先对最靠近当前柱面位置的请求进行服务，即先对寻找时间最短的请求进行服务。SSTF 算法总是让寻找时间最短的那个请求先服务，而不管请求访问者到来的先后次序。
- 3) SCAN 算法：读写头从盘的一端开始朝另一端移动，在移动中搜索每个磁道上的请求，若有则服务之，直至到达盘的另一端。在另一端，磁头移动的方向是相反的，并继续在移动中扫描服务。（电梯算法）
- 4) C-SCAN 算法：C-SCAN (Circular-SCAN) 同 SCAN 算法一样，C-SCAN 也将磁头从盘的一端移动到另一端，并在移动中对遇到的请求进行服务。所不同的是当它到达另一端时，它马上折回到盘的开始端，然而不对返回路径上的任何请求服务。

34、什么是 TLB?

TLB 的作用是在处理器访问内存数据的时候做地址转换。TLB 的全称是 Translation Lookaside Buffer，可以翻译做旁路缓冲，是一个具有并行查询能力的特殊高速缓冲寄存器。TLB 中存放了一些页表文件，文件中记录了虚拟地址和物理地址的映射关系。当应用程序访问一个虚拟地址的时候，会从 TLB 中查询出对应的物理地址，然后访问物理地址。TLB 通常是一个分层结构，使用与 Cache 类似的原理。处理器使用一定的算法把最常用的页表放在最先访问的层次。

35、作业和进程的区别

一个进程是一个程序对某个数据集的执行过程，是分配资源的基本单位。作业是用户需要计算机完成的某项任务，是要求计算机所做工作的集合。一个作业的完成要经过作业提交、作业收容、作业执行和作业完成 4 个阶段。而进程是对已提交完毕的程序所执行过程的描述，是资源分配的基本单位。

其主要区别如下。

(1)作业是用户向计算机提交任务的任务实体。在用户向计算机提交作业后，系统将它放入外存中的作业等待队列中等待执行。而进程则是完成用户任务的执行实体，是向系统申请分配资源的基本单位。任一进程，只要它被创建，总有相应的部分存在于内存中。

(2) 一个作业可由多个进程组成，且必须至少由一个进程组成，反过来则不成立。

(3) 作业的概念主要用在批处理系统中，像 UNIX 这样的分时系统中就没有作业的概念。而进程的概念则用在几乎所有的多道程序系统中进程是操作系统进行资源分配的单位。在 Windows 下,进程又被细化为线程,也就是一个进程下有多个能独立运行的更小的单位。

36、银行家算法

主要思想是避免系统进入不安全状态，在每次进行资源分配时，它首先检查系统是否有足够的资源满足要求，如果有，则先试行分配，并对分配后的新状态进行安全性检查。如果新状态安全，则正式分配上述资源，否则拒绝分配上述资源。这样就保证系统始终处于安全状态，从而避免死锁现象的发生。

37、RAID 磁盘阵列

独立磁盘冗余阵列（RAID，redundant array of independent disks，redundant array of inexpensive disks）是把相同的数据存储在多个硬盘的不同的地方（因此，冗余地）的方法。通过把数据放在多个硬盘上，输入输出操作能以平衡的方式交叠，改良性能。因为多个硬盘增加了平均故障间隔时间（MTBF），储存冗余数据也增加了容错。

3.3.5 计算机网络

1、介绍七层网络体系结构。

为什么分七层

支持异构网络的互联互通。

七层分别负责的内容（功能）

OSI 模型把网络通信的工作分为 7 层，从下到上分别是物理层、数据链路层、网络层、传输层、会话层、表示层和应用层。

(1) 物理层

任务：透明地传输比特流。

功能：为数据段设备提供传送数据通路

传输单位：比特

所实现的硬件：集线器，中继器

(2) 数据链路层

任务：将网络层传输下来的 IP 数据报组装成帧

功能：a. 链路连接的建立、拆除和分离

b. 帧定界和帧同步

c. 差错检测

传输单位：帧

所实现的硬件：交换机、网桥

协议：PPP, HDLC、SDLC、STP、ARQ

(3) 网络层

任务：a. 将传输层传下来的报文段封装成分组

b. 选择合适的路由，使得传输层传下来的分组能够交付到目的主机

功能：a. 为传输层提供服务

b. 组包和拆包

c. 路由选择

d. 拥塞控制

传输单位：数据段

所实现的硬件：路由器

协议：ICMP、ARP、RARP、IP、IGMP、OSPF

(4)传输层

任务：负责主机中两个进程之间的通信

功能：

- a. 为端到端连接提供可靠的服务
- b. 为端到端连接提供流量控制、差错控制、服务质量等管理服务

传输单位：报文段（TCP）或用户数据报（UDP）

协议：TCP、UDP

(5)会话层

任务：不同主机上各进程间的对话

功能：管理主机间的会话进程，包括建立、管理以及终止进程间的会话。是一种端到端的服务

(6)表示层

负责处理在两个内部数据表示结构不同的通信系统之间交换信息的表示格式，为数据加密和解密以及为提高传输效率提供必需的数据压缩以及解压等功能。

(7)应用层

任务：提供系统与用户的接口

功能：

- a. 文件传输
- b. 访问和管理
- c. 电子邮件服务

协议：FTP、SMTP、POP3、HTTP、DNS、TELnet

2、五层网络体系结构。

五层网络体系结构分分别为：应用层、运输层、网络层、数据链路层、物理层。

3、TCP、HTTP、FTP 分别属于哪一层？

TCP、HTTP、FTP 分别属于传输层、应用层、应用层。

4、ARP 协议的理解。

ARP 协议即地址解析协议，是根据 IP 地址获取 MAC 地址的一个网络层协议。

工作原理

ARP 首先会发起一个请求数据包，数据包的首部包含了目标主机的 IP 地址，然后这个数据包会在链路层进行再次包装，生成以太网数据包，最终由以太网广播给子网内的所有主机，每一台主机都会接收到这个数据包，并取出标头里的 IP 地址，然后和自己的 IP 地址进行比较，如果相同就返回自己的 MAC 地址，如果不同就丢弃该数据包。ARP 接收返回消息，以此确定目标机的 MAC 地址；与此同时，ARP 还会将返回的 MAC 地址与对应的 IP 地址存入本机 ARP 缓存中并保留一定时间，下次请求时直接查询 ARP 缓存以节约资源。

5、应用层都包含什么协议？

应用层包含的协议有 DNS、FTP、SMTP、HTTP、SNMP、Telnet 等，其作用分别如下：

域名系统(Domain Name System, DNS)：用于实现网络设备名字到 IP 地址映射的网络服务。

文件传输协议(File Transfer Protocol, FTP)：用于实现交互式文件传输功能。

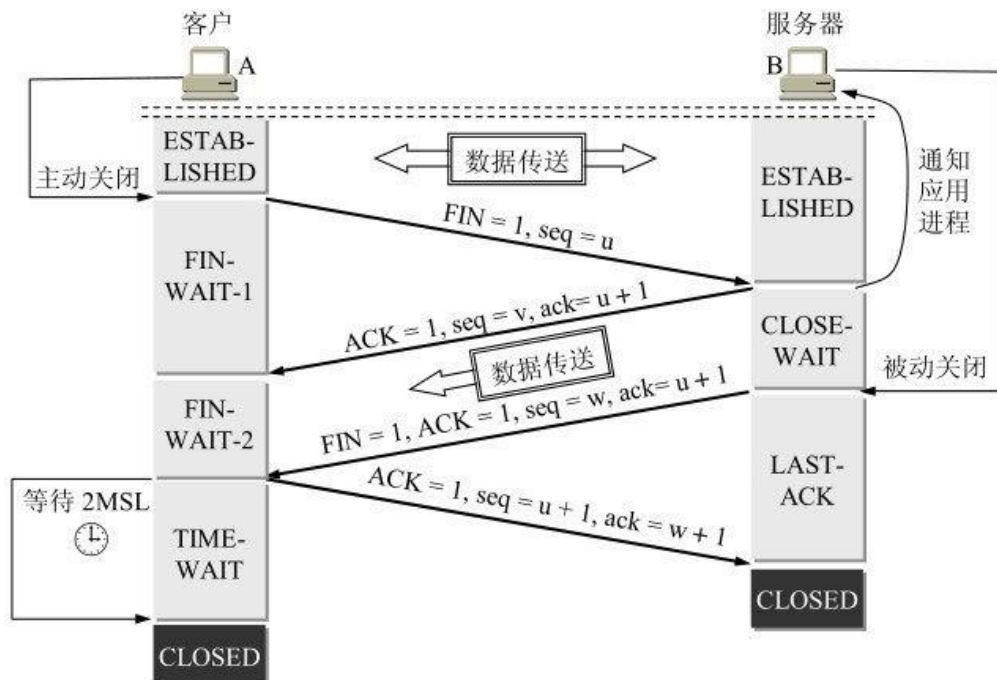
简单邮件传送协议(Simple Mail Transfer Protocol, SMTP)：用于实现电子邮箱传送功能。

超文本传输协议(HyperText Transfer Protocol, HTTP)：用于实现 WWW 服务。

简单网络管理协议(simple Network Management Protocol, SNMP)：用于管理与监视网络设备。

远程登录协议(Telnet)：用于实现远程登录功能。

6、介绍一下 tcp 的四次挥手。



客户端进程发出连接释放报文，并且停止发送数据。释放数据报文首部， $FIN=1$ ，其序列号为 $seq=u$ （等于前面已经传送过来的数据的最后一个字节的序号加 1），此时，客户端进入 **FIN-WAIT-1**（终止等待 1）状态。TCP 规定，**FIN** 报文段即使不携带数据，也要消耗一个序号。

服务器收到连接释放报文，发出确认报文， $ACK=1, ack=u+1$ ，并且带上自己的序列号 $seq=v$ ，此时，服务端就进入了 **CLOSE-WAIT**（关闭等待）状态。TCP 服务器通知高层的应用进程，客户端向服务器的方向就释放了，这时候处于半关闭状态，即客户端已经没有数据要发送了，但是

服务器若发送数据，客户端依然要接受。这个状态还要持续一段时间，也就是整个 **CLOSE-WAIT** 状态持续的时间。

客户端收到服务器的确认请求后，此时，客户端就进入 **FIN-WAIT-2**（终止等待 2）状态，等待服务器发送连接释放报文（在这之前还需要接受服务器发送的最后的的数据）。

服务器将最后的数据发送完毕后，就向客户端发送连接释放报文， $FIN=1, ack=u+1$ ，由于在半关闭状态，服务器很可能又发送了一些数据，假定此时的序列号为 $seq=w$ ，此时，服务器就进入了 **LAST-ACK**（最后确认）状态，等待客户端的确认。

客户端收到服务器的连接释放报文后，必须发出确认， $ACK=1, ack=w+1$ ，而自己的序列号是 $seq=u+1$ ，此时，客户端就进入了 **TIME-WAIT**（时间等待）状态。注意此时 TCP 连接还没有释放，必须经过 $2 * MSL$ （最长报文段寿命）的时间后，当客户端撤销相应的 TCB 后，才进入 **CLOSED** 状态。

服务器只要收到了客户端发出的确认, 立即进入 CLOSED 状态。同样, 撤销 TCB 后, 就结束了这次的 TCP 连接。可以看到, 服务器结束 TCP 连接的时间要比客户端早一些。

7、为什么需要四次挥手?

TCP 协议是一种面向连接的、可靠的、基于字节流的运输层通信协议。TCP 是全双工模式, 这就意味着, 当客户端发出 FIN 报文段时, 只是表示客户端已经没有数据要发送了, 客户端告诉服务器, 它的数据已经全部发送完毕了; 但是, 这个时候客户端还是可以接受来自服务端的数据; 当服务端返回 ACK 报文段时, 表示它已经知道客户端没有数据发送了, 但是服务端还是可以发送数据到客户端的; 当服务端也发送了 FIN 报文段时, 这个时候就表示服务端也没有数据要发送了, 就会告诉客户端, 我也没有数据要发送了, 之后彼此就会愉快的中断这次 TCP 连接。

简单地说, 前 2 次挥手用于关闭一个方向的数据通道, 后两次挥手用于关闭另外一个方向的数据通道。

8、为什么要有最后一次 ACK

客户端首先向服务器发送一个连接请求, 但是可能这个连接请求走了远路, 等了很长时间, 服务器都没有收到, 那么客户端可能会再次发送, 此时服务器端收到并且回复 SYN、ACK; 在这个时候最先发送的那个连接请求到达服务器, 那么服务器会回复一个 SYN, ACK; 但是客户端表示自己已经收到确认了, 并不搭理这个回复, 那么服务器可能陷入等待, 如果这种情况多了, 那么会导致服务器瘫痪, 所以要发送第三个确认。

9、介绍一下 tcp 粘包、拆包的机制。

TCP 粘包和拆包问题

TCP 是一个“流”协议, 所谓流, 就是没有界限的一长串二进制数据。TCP 作为传输层协议并不了解上层业务数据的具体含义, 它会根据 TCP 缓冲区的实际情况进行数据包的划分, 所以在业务上认为是一个完整的包, 可能会被 TCP 拆分成多个包进行发送, 也有可能把多个小的包封装成一个大的数据包发送, 这就是所谓的 TCP 粘包和拆包问题。

产生 TCP 粘包和拆包的原因

我们知道 TCP 是以流动的方式传输数据的, 传输的最小单位为一个报文段 (Segment)。

TCP Header 中有个 Options 标识位。常见的标识位为 MSS (Maximum Segment Size) 指的是, 连接层每次传输的数据有个最大限制 MTU (Maximum Transmission Unit), 一般是 1500bit, 超过这个量要分成多个报文段, MSS 则是这个最大限制减去 TCP 的 header, 光是要传输的数据的大小, 一般为 1460bit。换算成字节, 也就是 180 多字节。TCP 为提高性能, 发送端会将需要发送的数据发送到缓冲区, 等待缓冲区满了以后, 再将缓冲中的数据发送到接收方。同理, 接收方也有缓冲区这样的机制来接受数据。发生 TCP 粘包、拆包主要是以下原因:

(1) 应用程序写入数据大于套接字缓冲区大小, 会发生拆包;

(2) 应用程序写入数据小于套接字缓冲区大小, 网卡将应用多次写入的数据发送到网络上, 这将会发送粘包;

(3) 进行 MSS(最大报文长度)大小的 TCP 分段, 当 TCP 报文长度——TCP header 长度 > MSS 的时候会发生拆包;

(4) 接收方法不及时读取套接字缓冲区数据, 这将发生粘包。

10、UDP 会不会产生粘包问题呢?

TCP 为了保证可靠传输并减少额外的开销 (每次发包都要验证), 采用了基于流的传输, 基于流的传输不认为消息是一条一条的, 是无保护消息边界的 (保护消息边界: 指传输协议把数据当做一条独立的消息在网上传输, 接收端一次只能接受一条独立的消息)。UDP 则是面向消息传输的, 是有保护消息边界的, 接收方一次只接受一条独立的信息, 所以不存在粘包问题。

11、介绍一下 TCP 和 UDP 的区别。

TCP 和 UDP 有如下区别:

连接: TCP 面向连接的传输层协议, 即传输数据之前必须先建立好连接; UDP 无连接。

服务对象: TCP 点对点的两点间服务, 即一条 TCP 连接只能有两个端点; UDP 支持一对一, 一对多, 多对一, 多对多的交互通信。

可靠性: TCP 可靠交付: 无差错, 不丢失, 不重复, 按序到达; UDP 尽最大努力交付, 不保证可靠交付。

拥塞控制/流量控制: 有拥塞控制和流量控制保证数据传输的安全性; UDP 没有拥塞控制, 网络拥塞不会影响源主机的发送效率。

报文长度: TCP 动态报文长度, 即 TCP 报文长度是根据接收方的窗口大小和当前网络拥塞

情况决定的；UDP 面向报文，不合并，不拆分，保留上面传下来报文的边界。

首部开销：TCP 首部开销大，首部 20 个字节；UDP 首部开销小，8 字节（源端口，目的端口，数据长度，校验和）。

适用场景（由特性决定）：数据完整性需让位于通信实时性，则应该选用 TCP 协议（如文件传输、重要状态的更新等）；反之，则使用 UDP 协议（如视频传输、实时通信等）。

12、为什么需要可靠的 UDP

在弱网（2G、3G、信号不好）环境下，使用 TCP 连接的延迟很高，影响体验。使用 UDP 是很好的解决方案，既然把 UDP 作为弱网里面的 TCP 来使用，就必须保证数据传输能像 TCP 一样可靠。

13、如何实现可靠的 UDP

UDP 它不属于连接型协议，因而具有资源消耗小，处理速度快的优点，所以通常音频、视频和普通数据在传送时使用 UDP 较多，因为它们即使偶尔丢失一两个数据包，也不会对接收结果产生太大影响。传输层无法保证数据的可靠传输，只能通过应用层来实现了。实现的方式可以参照 tcp 可靠性传输的方式，只是实现不在传输层，实现转移到了应用层。关键在于两点，从应用层角度考虑：

- （1）提供超时重传，能避免数据报丢失。
- （2）提供确认序列号，可以对数据报进行确认和排序。

本端：首先在 UDP 数据报定义一个首部，首部包含确认序列号和时间戳，时间戳是用来计算 RTT(数据报传输的往返时间)，计算出合适的 RTO(重传的超时时间)。然后以等-停的方式发送数据报，即收到对端的确认之后才发送下一个的数据报。当时间超时，本端重传数据报，同时 RTO 扩大为原来的两倍，重新开始计时。

对端：接受到一个数据报之后取下该数据报首部的时间戳和确认序列号，并添加本端的确认数据报首部之后发送给对端。根据此序列号对已收到的数据报进行排序并丢弃重复的数据报。

14、TCP 报文首部中序号占多少字节？

序号字段占 4 个字节（32 位）。

15、TCP 是怎么控制流量的？

所谓流量控制就是让发送速率不要过快，让接收方来得及接收。

TCP 控制流量的方法

利用滑动窗口机制就可以实施流量控制。

原理就是运用 TCP 报文段中的窗口大小字段来控制，发送方的发送窗口不可以大于接收方发回的窗口大小。考虑一种特殊的情况，就是接收方若没有缓存足够使用，就会发送零窗口大小的报文，此时发送方将发送窗口设置为 0，停止发送数据。之后接收方有足够的缓存，发送了非零窗口大小的报文，但是这个报文在中途丢失的，那么发送方的发送窗口就一直为零导致死锁。

解决这个问题，TCP 为每一个连接设置一个持续计时器（persistence timer）。只要 TCP 的一方收到对方的零窗口通知，就启动该计时器，周期性的发送一个零窗口探测报文段。对方就在确认这个报文的时候给出现在的窗口大小（注意：TCP 规定，即使设置为零窗口，也必须接收以下几种报文段：零窗口探测报文段、确认报文段和携带紧急数据的报文段）。

16、TCP 如何保证可靠性？

TCP 协议保证数据传输可靠性的方式主要有：校验和、序列号、确认应答、超时重传、连接管理、流量控制、拥塞控制。

17、TCP 里的 reset 状态。

TCP 异常终止（reset 报文）

TCP 的异常终止是相对于正常释放 TCP 连接的过程而言的，我们都知道，TCP 连接的建立是通过三次握手完成的，而 TCP 正常释放连接是通过四次挥手来完成，但是有些情况下，TCP 在交互的过程中会出现一些意想不到的情况，导致 TCP 无法按照正常的四次挥手来释放连接，如果此时不通过其他方式来释放 TCP 连接的话，这个 TCP 连接将会一直存在，占用系统的部分资源。在这种情况下，我们就需要有一种能够释放 TCP 连接的机制，这种机制就是 TCP 的 reset 报文。reset 报文是指 TCP 报头的标志字段中的 reset 位置一的报文。

RST 标志位（Reset）

RST 表示复位，用来异常的关闭连接，在 TCP 的设计中它是不可或缺的。就像上面说的一

样, 发送 RST 包关闭连接时, 不必等缓冲区的包都发出去 (不像上面的 FIN 包), 直接就丢弃缓存区的包发送 RST 包。而接收端收到 RST 包后, 也不必发送 ACK 包来确认。

TCP 处理程序会在自己认为的异常时刻发送 RST 包。例如, A 向 B 发起连接, 但 B 之上并未监听相应的端口, 这时 B 操作系统上的 TCP 处理程序会发 RST 包。

又比如, AB 正常建立连接了, 正在通讯时, A 向 B 发送了 FIN 包要求关连接, B 发送 ACK 后, 网断了, A 通过若干原因放弃了这个连接 (例如进程重启)。网通了后, B 又开始发数据包, A 收到后表示压力很大, 不知道这野连接哪来的, 就发了个 RST 包强制把连接关了, B 收到后会出现 connect reset by peer 错误。

18、报文乱序怎么办?

数据包会因为 IP 层所规划的路由链路的不同导致数据包的接收顺序与发送顺序会有所不同。另外因为 TCP 是一种全双工的协议, 乱序可能发生在正向链路, 也可能发生在反向链路, 这两种不同的情况给 TCP 带来的影响也会略有差异。

正向链路乱序

此时 TCP 会无法判断是数据包丢失还是乱序, 因为丢包和乱序都会导致接收端收到次序混乱的数据包, 造成接收端的数据空洞。TCP 会将这种情况暂定为数据包的乱序, 因为乱序是时间问题 (可能是数据包的迟到), 而丢包则意味着重传。当 TCP 意识到包出现乱序的情况时, 会立即 ACK, 该 ACK 的 TSEQ 部分包含的 TSEQ 值会记录当前接收端收到有序报文段的时刻。这会使得数据包的 RTT 样本值增大, 进一步导致 RTO 时间延长。这对 TCP 来说无疑是有益的, 因为 TCP 有充分的时间判断数据包到底是失序还是丢了来防止不必要的数据重传。当然严重的乱序则会让发送端以为是丢包一旦重复的 ACK 超过 TCP 的阈值, 便会触发超时重传机制, 以及时解决这种问题。

反向链路 (ACK) 乱序

顾名思义, 如果发生这种情况, 就会导致发送端窗口快速前移, 这会导致发送端出现不必要的流量突发, 影响网络带宽。

19、IP 为什么要分类?

根据 IP 地址访问终端是通过路由器, 路由设备当中有一张路由表, 该路由表记录了所有 IP 地址的位置, 这样就可以进行包的转发了, 如果我们不区分网络地址, 那么这张路由表当中

就要保存有所有 IP 地址的方向，这张路由表就会很大，就像下面说的那样：如果不分网络位和主机位，路由器的路由表就是都是 32 位的地址，那所有的路由器维护的路由表会很大，转发速度会变慢（因为查询变慢）。而且所有的路由器都要有全 Internet 的地址，所有人的路由器都要有足够的性能来存下全网地址。估计建造这样的 Internet 成本是现在的几万倍，甚至更高。

有了网络地址，就可以限定拥有相同网络地址的终端都在同一个范围内，那么路由表只需要维护这个网络地址的方向，就可以找到相应的终端了。

20、IPV4 和 IPV6 有什么区别？

IPv4 和 IPv6 是是目前使用的两种 Internet 协议版本，IPv4 和 IPv6 协议之间存在各种差异，包括它们的功能，但关键的一点是它生成的地址（地址空间）的数量的区别。

21、说一下 http 和 https 的区别。

https 和 http 主要存在以下的区别：

HTTPS 协议需要到 CA（Certificate Authority，证书颁发机构）申请证书，一般免费证书较少，因而需要一定费用。（以前的网易官网是 http，而网易邮箱是 https。）

HTTP 是超文本传输协议，信息是明文传输，HTTPS 则是具有安全性的 SSL 加密传输协议。

HTTP 和 HTTPS 使用的是完全不同的连接方式，用的端口也不一样，前者是 80，后者是 443。

HTTP 的连接很简单，是无状态的。HTTPS 协议是由 SSL+HTTP 协议构建的可进行加密传输、身份认证的网络协议，比 HTTP 协议安全。（无状态的意思是其数据包的发送、传输和接收都是相互独立的。无连接的意思是指通信双方都不长久的维持对方的任何信息。）

22、https 为什么采用混合加密机制？

一方面，第一阶段的非对称加密，保证了对称密钥的安全性；另一方面，第二阶段的对称加密，可以提高加密/解密处理的速度，提高数据传输的效率。

23、https 支持什么加密算法？

常见的对称加密算法有：DES、3DES、Blowfish、IDEA、RC4、RC5、RC6 和 AES；

常见的非对称加密算法有：RSA、ECC（移动设备用）、Diffie-Hellman、El Gamal、DSA（数字签名用）；

常见的 Hash 算法有：MD2、MD4、MD5、HAVAL、SHA；

24、HTTP 请求头中包含什么内容？

HTTP 请求头中包含如下三个内容：

User-Agent: 产生请求的浏览器类型。

Accept: 客户端可识别的内容类型列表。

Host: 主机地址。

25、HTTP 是基于 TCP 还是 UDP？

HTTP 是基于 TCP 的。

26、介绍一下 HTTP 协议中的长连接和短连接。

HTTP 协议的底层使用 TCP 协议，所以 HTTP 协议的长连接和短连接在本质上是 TCP 层的长连接和短连接。由于 TCP 建立连接、维护连接、释放连接都是要消耗一定的资源，浪费一定的时间。对于服务器来说，频繁的请求释放连接会浪费大量的时间，长时间维护太多的连接的话又需要消耗资源。所以长连接和短连接并不存在优劣之分，只是适用的场合不同而已。长连接和短连接分别有如下优点和缺点：

长连接优点：可以节省较多的 TCP 连接和释放的操作，节约时间，对于频繁请求资源的用户来说，适合长连接。

长连接缺点：由于有保活功能，当遇到大量的恶意连接时，服务器的压力会越来越大。这时服务器需要采取一些策略，关闭一些长时间没有进行读写事件的连接。

短连接优点：短连接对服务器来说管理比较简单，只要存在的连接都是有效连接，不需要额外的控制手段，而且不会长时间占用资源。

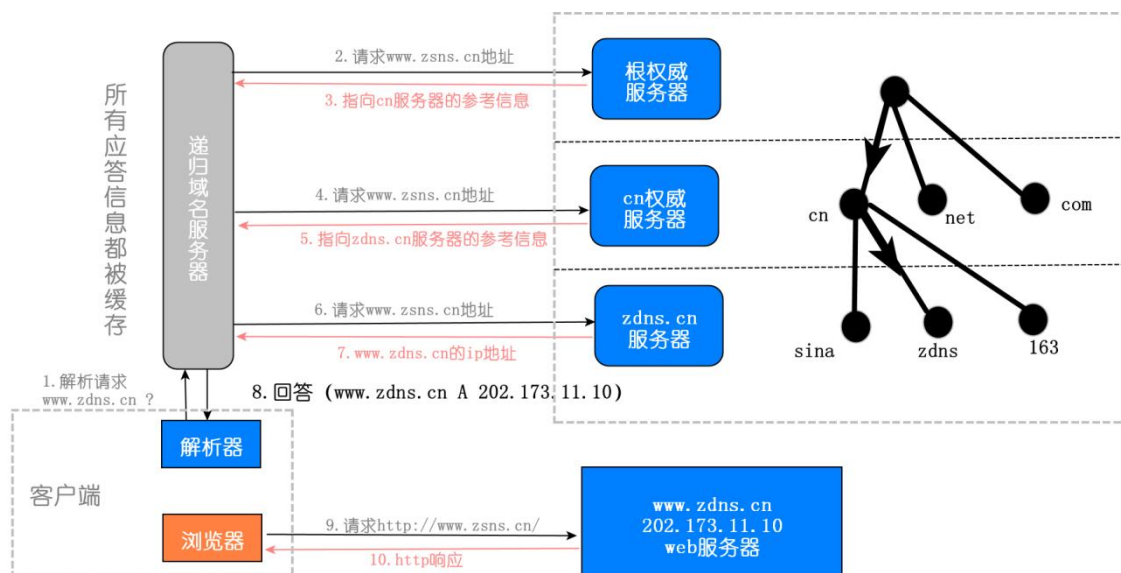
短连接缺点：如果客户端请求频繁的话，会在 TCP 的建立和释放上浪费大量的时间。

27、什么是重定向？

重定向(Redirect)就是通过各种方法将各种网络请求重新定个方向转到其它位置（如：网

页重定向、域名的重定向、路由选择的变化也是对数据报文经由路径的一种重定向)。

28、介绍一下 DNS 寻址的过程。



首先客户端位置是一台电脑或手机，在打开浏览器以后，比如输入 `http://www.zdns.cn` 的域名，它首先是由浏览器发起一个 DNS 解析请求，如果本地缓存服务器中找不到结果，则首先会向根服务器查询，根服务器里面记录的都是各个顶级域所在的服务器的位置，当向根服务器请求 `http://www.zdns.cn` 的时候，根服务器就会返回 `.cn` 服务器的位置信息；

递归服务器拿到 `.cn` 的权威服务器地址以后，就会寻问 `.cn` 的权威服务器，知不知道 `http://www.zdns.cn` 的位置。这个时候 `cn` 权威服务器查找并返回 `http://zdns.cn` 服务器的地址；

继续向 `http://zdns.cn` 的权威服务器去查询这个地址，由 `http://zdns.cn` 的服务器给出了地址：202.173.11.10；

最终进入 `http` 的链接，顺利访问网站；

29、对 TIME_WAIT 的理解。

出现 TIME_WAIT 的状态原因

TIME_WAIT 状态之所以存在,是为了保证网络的可靠性。由于 TCP 连接是双向的,所以在关闭连接的时候,两个方向各自都需要关闭。先发 FIN 包的一方执行的是主动关闭,后发送 FIN 包的一方执行的是被动关闭。主动关闭的一方会进入 TIME_WAIT 状态,并且在此状态停留 2MSL

时长。如果 Server 端一直没有向 client 端发送 FIN 消息(调用 close() API), 那么这个 CLOSE_WAIT 会一直存在下去。

MSL 概念

其指的是报文段的最大生存时间。如果报文段在网络中活动了 MSL 时间, 还没有被接收, 那么就会被丢弃。关于 MSL 的大小, RFC 793 协议中给出的建议是 2 分钟, 不过 Linux 中, 通常是半分钟。

TIME_WAIT 持续两个 MSL 的作用

首先, 可靠安全地关闭 TCP 连接。比如网络拥塞, 如果主动关闭方最后一个 ACK 没有被被动关闭方接收到, 这时被动关闭方会对 FIN 进行超时重传, 在这时尚未关闭的 TIME_WAIT 就会把这些尾巴问题处理掉, 不至于对新连接及其他服务产生影响。其次, 防止由于没有持续 TIME_WAIT 时间导致的新的 TCP 连接建立起来, 延迟的 FIN 重传包会干扰新的连接。

TIME_WAIT 占用的资源

少量内存 (大概 4K) 和一个文件描述符 fd。

TIME_WAIT 关闭的危害

首先, 当网络情况不好时, 如果主动方无 TIME_WAIT 等待, 关闭前个连接后, 主动方与被动方又建立起新的 TCP 连接, 这时被动方重传或延时过来的 FIN 包到达后会直接影响新的 TCP 连接; 其次, 当网络情况不好时, 同时没有 TIME_WAIT 等待时, 关闭连接后无新连接, 那么当接收到被动方重传或延迟的 FIN 包后, 会给被动方回送一个 RST 包, 可能会影响被动方其他的连接。

30、TCP 拥塞控制解决方法

TCP 拥塞控制的四种算法: 慢开始、拥塞避免、快重传、快恢复。

(1) 慢开始算法: 当主机开始发送数据时, 并不清楚网络的负载情况, 所以由小到大逐渐增大拥塞窗口, 每经过一个传输轮次没有出现超时就将拥塞窗口加倍。同时还需要设置一个慢开始门限, 在拥塞窗口小于慢开始门限时使用慢开始算法, 大于慢开始门限时, 使用拥塞避免算法;

(2) 拥塞避免算法: 在拥塞窗口大于慢开始门限时, 让拥塞窗口按线性规律缓慢增长。即每经过一个传输轮次, 拥塞窗口增大一个 MSS 最大报文段尺寸。(拥塞避免并非完全能够避免拥塞, 只是使网络比较不容易出现拥塞)

(3) 快重传算法：使发送方今早知道发生了个别报文段丢失，并不是出现网络拥塞。

要求接受不要等自己发送数据时才进行捎带确认，而是立即发送确认，即使收到了失序的报文段也要立即发出对已收到报文段的重复确认。而发送方一旦受到三个连续的重读确认，就将相应的报文段立即重传。

(4) 快恢复算法：发送方知道只有个别报文段丢失而不是网络拥塞时，不启动慢开始算法，而是执行快恢复算法，将慢开始门限和拥塞窗口值调整为当前窗口的一半，开始执行拥塞避免算法

31、数据链路层的作用

数据链路层在物理层提供的服务的基础上向网络层提供服务，将原始的、有差错的物理线路改进成逻辑上无差错的数据链路，从而向网络层提供高质量的服务。功能主要有：①链路管理；②封装成帧；③透明传输；④差错检测

32、路由协议有哪些

域内：

①RIP（路由信息协议）。距离矢量协议；16 跳；收敛速度慢；适用小规模网络；仅和邻居交换信息，且交换的是整个路由表，按固定时间周期性交换；实现简单，开销小；传输层使用 UDP；RIP 报文限制大小为 512 字节。

②OSPF（开放最短路由协议）。链路状态协议；基于 IP；向自治域中所有路由器发送信息（防洪法），且发送的是链路状态表（LSA），触发更新；更新过程收敛快；根据代价选择最佳路由；支持多路径负载均衡。

域间：

BGP（外部网关协议）是不同 AS 之间的路由器之间交换路由信息的协议。

33、频分复用 时分复用 波分复用 码分复用

频分复用：给每个信号分配唯一的载波频率并通过单一媒体来传输多个独立信号的方法。

时分复用：把多个信号复用到单个硬件传输信道，它允许每个信号在一个很短的时间使用信道，接着再让下一个信号使用。

波分复用：就是光的频分复用。用一根光纤同时传输多个频率很接近的光载波信号。

码分复用：码分复用是用一组包含互相正交的码字的码组携带多路信号。每一个用户可以在同样的时间使用同样的频带进行通信。由于各用户使用经过特殊挑选的不同码型，各用户之间不会造成干扰，因此这种系统发送的信号有很强的抗干扰能力。

34、说一下路由器的原理

传统地，路由器工作于 OSI 七层协议中的第三层，其主要任务是接收来自一个网络接口的数据包，根据其中所含的目的地址，决定转发到下一个目的地址。因此，路由器首先去掉数据包的二层头，取出目的 IP 地址，在转发路由表中查找它对应的下一跳地址，若找到，就在数据包的帧格前添加下一个 MAC 地址，同时 IP 数据包头的 TTL（Time To Live）域也减一，并重新计算校验和。当数据包被送到输出端口时，它需要按顺序等待，以便被传送到输出链路上。

35、分组交换的优点和缺点

与报文交换相比较，分组交换的优点和缺点如下。

优点：

① 加速传输：因为分组交换是逐个传输，所以可以使后一个分组的存储操作与前一个分组的转发操作并行，这种流水线式传输方式减少了报文的传输时间，此外，传输一个分组所需的缓冲区比传输一个报文所需的缓冲区小得多，这样因缓冲区不足而等待发送的几率及等待的时间也必然减少。

② 简化了存储管理：分组的长度固定，相应的缓冲区大小也固定。

③ 减少出错几率和重发数据量：因为分组较短，其出错几率必然减少，所以每次重发的数据量也就大大减少，这样不仅提高了可靠性，也减少了传输时延。

缺点：

① 存在传输时延：尽管与报文交换相比的传输时延较短，但是与电路交换相比仍然存在存储转发时延，而且对其结点交换机的处理能力有更高的要求。

② 当分组交换采用数据报服务时，可能出现失序、丢失或重复分组，分组到达的目的地的结点时，要对分组按编号进行排序工作，工作量较大。

3.3.6 数据库

1、数据库的左连接和右连接

外连接通过 `OUTER JOIN` 来实现，它会返回两张表中满足连接条件的数据，同时返回不满足连接条件的数据。常见的外连接有两种形式：左外连接（`LEFT OUTER JOIN`）、右外连接（`RIGHT OUTER JOIN`）。

左外连接：可以简称为左连接（`LEFT JOIN`），它会返回左表中的所有记录和右表中满足连接条件的记录。

右外连接：可以简称为右连接（`RIGHT JOIN`），它会返回右表中的所有记录和左表中满足连接条件的记录。

2、谈谈你对 SQL 注入的理解

SQL 注入的原理是将 SQL 代码伪装到输入参数中，传递到服务器解析并执行的一种攻击手法。也就是说，在一些对 `SERVER` 端发起的请求参数中植入一些 SQL 代码，`SERVER` 端在执行 SQL 操作时，会拼接对应参数，同时也将一些 SQL 注入攻击的“SQL”拼接起来，导致会执行一些预期之外的操作。

举个例子：

比如我们的登录功能，其登录界面包括用户名和密码输入框以及提交按钮，登录时需要输入用户名和密码，然后提交。此时调用接口 `/user/login/` 加上参数 `username`、`password`，首先连接数据库，然后后台对请求参数中携带的用户名、密码进行参数校验，即 SQL 的查询过程。假设正确的用户名和密码为 `ls` 和 `123456`，输入正确的用户名和密码、提交，相当于调用了以下的 SQL 语句。

```
SELECT * FROM user WHERE username = 'ls' AND password = '123456'
```

SQL 中会将 `#` 及 `--` 以后的字符串当做注释处理，如果我们使用 `' or 1=1 #` 作为用户名参数，那么服务端构建的 SQL 语句就如下：

```
select * from user where username=' or 1=1 #' and password='123456'
```

而 `#` 会忽略后面的语句，而 `1=1` 属于常等型条件，因此这个 SQL 将查询出所有的登录用户。其实上面的 SQL 注入只是在参数层面做了些手脚，如果是引入了一些功能性的 SQL 那就更危险了，比如上面的登录功能，如果用户名使用这个 `' or 1=1;delete * from users; #`，那么在 `"` 之后

相当于是另外一条新的 SQL，这个 SQL 是删除全表，是非常危险的操作，因此 SQL 注入这种还是需要特别注意的。

3、如何解决 SQL 注入

严格的参数校验

参数校验就没得说了，在一些不该有特殊字符的参数中提前进行特殊字符校验即可。

SQL 预编译

在知道了 SQL 注入的原理之后，我们同样也了解到 MySQL 有预编译的功能，指的是在服务器启动时，MySQL Client 把 SQL 语句的模板（变量采用占位符进行占位）发送给 MySQL 服务器，MySQL 服务器对 SQL 语句的模板进行编译，编译之后根据语句的优化分析对相应的索引进行优化，在最终绑定参数时把相应的参数传送给 MySQL 服务器，直接进行执行，节省了 SQL 查询时间，以及 MySQL 服务器的资源，达到一次编译、多次执行的目的，除此之外，还可以防止 SQL 注入。

具体是怎样防止 SQL 注入的呢？实际上当将绑定的参数传到 MySQL 服务器，MySQL 服务器对参数进行编译，即填充到相应的占位符的过程中，做了转义操作。我们常用的 JDBC 就有预编译功能，不仅提升性能，而且防止 SQL 注入。

4、WHERE 和 HAVING 有什么区别？

WHERE 是一个约束声明，使用 WHERE 约束来自数据库的数据，WHERE 是在结果返回之前起作用的，WHERE 中不能使用聚合函数。

HAVING 是一个过滤声明，是在查询返回结果集以后对查询结果进行的过滤操作，在 HAVING 中可以使用聚合函数。另一方面，HAVING 子句中不能使用除了分组字段和聚合函数之外的其他字段。

从性能的角度来说，HAVING 子句中如果使用了分组字段作为过滤条件，应该替换成 WHERE 子句。因为 WHERE 可以在执行分组操作和计算聚合函数之前过滤掉不需要的数据，性能会更好。

5、对 MySQL 索引的理解

索引是一个单独的、存储在磁盘上的数据库结构，包含着对数据表里所有记录的引用指针。

使用索引可以快速找出在某个或多个列中有一特定值的行，所有 MySQL 列类型都可以被索引，对相关列使用索引是提高查询操作速度的最佳途径。

索引是在存储引擎中实现的，因此，每种存储引擎的索引都不一定完全相同，并且每种存储引擎也不一定支持所有索引类型。MySQL 中索引的存储类型有两种，即 BTREE 和 HASH，具体和表的存储引擎相关。MyISAM 和 InnoDB 存储引擎只支持 BTREE 索引；MEMORY/HEAP 存储引擎可以支持 HASH 和 BTREE 索引。

6、索引的优缺点

索引的优点主要有以下几条：

通过创建唯一索引，可以保证数据库表中每一行数据的唯一性。

可以大大加快数据的查询速度，这也是创建索引的主要原因。

在实现数据的参考完整性方面，可以加速表和表之间的连接。

在使用分组和排序子句进行数据查询时，也可以显著减少查询中分组和排序的时间。

增加索引也有许多不利的方面，主要表现在如下几个方面：

创建索引和维护索引要耗费时间，并且随着数据量的增加所耗费的时间也会增加。

索引需要占磁盘空间，除了数据表占数据空间之外，每一个索引还要占一定的物理空间，如果有大量的索引，索引文件可能比数据文件更快达到最大文件尺寸。

当对表中的数据进行增加、删除和修改的时候，索引也要动态地维护，这样就降低了数据的维护速度。

7、索引有哪几种？

MySQL 的索引可以分为以下几类：

普通索引和唯一索引

普通索引是 MySQL 中的基本索引类型，允许在定义索引的列中插入重复值和空值。

唯一索引要求索引列的值必须唯一，但允许有空值。如果是组合索引，则列值的组合必须唯一。

主键索引是一种特殊的唯一索引，不允许有空值。

单列索引和组合索引

单列索引即一个索引只包含单个列，一个表可以有多个单列索引。

组合索引是指在表的多个字段组合上创建的索引，只有在查询条件中使用了这些字段的左边字段时，索引才会被使用。使用组合索引时遵循最左前缀集合。

全文索引

全文索引类型为 **FULLTEXT**，在定义索引的列上支持值的全文查找，允许在这些索引列中插入重复值和空值。全文索引可以在 **CHAR**、**VARCHAR** 或者 **TEXT** 类型的列上创建。

空间索引

空间索引是对空间数据类型的字段建立的索引，MySQL 中的空间数据类型有 4 种，分别是 **GEOMETRY**、**POINT**、**LINestring** 和 **POLYGON**。MySQL 使用 **SPATIAL** 关键字进行扩展，使得能够用创建正规索引类似的语法创建空间索引。创建空间索引的列，必须将其声明为 **NOT NULL**，空间索引只能在存储引擎为 **MyISAM** 的表中创建。

8、MySQL 怎么判断要不要加索引？

建议按照如下的原则来创建索引：

当唯一性是某种数据本身的特征时，指定唯一索引。使用唯一索引需能确保定义的列的数据完整性，以提高查询速度。

在频繁进行排序或分组（即进行 **group by** 或 **order by** 操作）的列上建立索引，如果待排序的列有多个，可以在这些列上建立组合索引。

9、只要创建了索引，就一定会走索引吗？

不一定。

比如，在使用组合索引的时候，如果没有遵从“最左前缀”的原则进行搜索，则索引是不起作用的。

举例，假设在 **id**、**name**、**age** 字段上已经成功建立了一个名为 **Multidx** 的组合索引。索引行中按 **id**、**name**、**age** 的顺序存放，索引可以搜索 **id**、**(id,name)**、**(id, name, age)** 字段组合。如果列不构成索引最左面的前缀，那么 MySQL 不能使用局部索引，如 **(age)** 或者 **(name,age)** 组合则不能使用该索引查询。

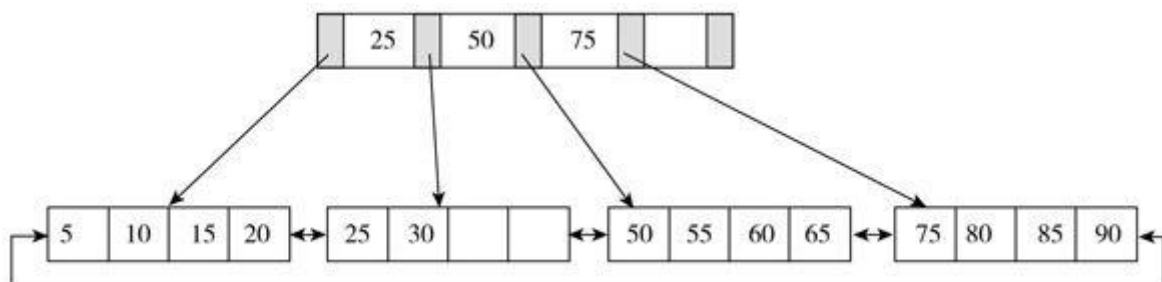
10、索引是越多越好吗？

索引并非越多越好，一个表中如有大量的索引，不仅占用磁盘空间，还会影响 **INSERT**、

DELETE、UPDATE 等语句的性能，因为在表中的数据更改时，索引也会进行调整和更新。

11、MySQL 的索引为什么用 B+树？

B+树由 B 树和索引顺序访问方法演化而来，它是为磁盘或其他直接存取辅助设备设计的一种平衡查找树，在 B+树中，所有记录节点都是按键值的大小顺序存放在同一层的叶子节点，各叶子节点通过指针进行链接。如下图：



B+树索引在数据库中的一个特点就是高扇出性，例如在 InnoDB 存储引擎中，每个页的大小为 16KB。在数据库中，B+树的高度一般都在 2~4 层，这意味着查找某一键值最多只需要 2 到 4 次 IO 操作，这还不错。因为现在一般的磁盘每秒至少可以做 100 次 IO 操作，2~4 次的 IO 操作意味着查询时间只需 0.02~0.04 秒。

12、聚簇索引和非聚簇索引有什么区别？

在 InnoDB 存储引擎中，可以将 B+树索引分为聚簇索引和辅助索引（非聚簇索引）。无论是何种索引，每个页的大小都为 16KB，且不能更改。

聚簇索引是根据主键创建的一棵 B+树，聚簇索引的叶子节点存放了表中的所有记录。辅助索引是根据索引键创建的一棵 B+树，与聚簇索引不同的是，其叶子节点仅存放索引键值，以及该索引键值指向的主键。也就是说，如果通过辅助索引来查找数据，那么当找到辅助索引的叶子节点后，很有可能还需要根据主键值查找聚簇索引来得到数据，这种查找方式又被称为书签查找。因为辅助索引不包含行记录的所有数据，这就意味着每页可以存放更多的键值，因此其高度一般都要小于聚簇索引。

13、什么是联合索引？

联合索引是指对表上的多个列进行索引，联合索引的创建方法与单个索引创建的方法一样，不同之处仅在于有多个索引列。从本质上来说，联合索引还是一棵 B+树，不同的是联合

索引的键值数量不是 1，而是大于等于 2，参考下图。另外，只有在查询条件中使用了这些字段的左边字段时，索引才会被使用，所以使用联合索引时遵循最左前缀集合。

14、说一说你对数据库事务的了解

事务可由一条非常简单的 SQL 语句组成，也可以由一组复杂的 SQL 语句组成。在事务中的操作，要么都执行修改，要么都不执行，这就是事务的目的，也是事务模型区别于文件系统的重要特征之一。

事务需遵循 ACID 四个特性：

A (atomicity)，原子性。原子性指整个数据库事务是不可分割的工作单位。只有使事务中所有的数据库操作都执行成功，整个事务的执行才算成功。事务中任何一个 SQL 语句执行失败，那么已经执行成功的 SQL 语句也必须撤销，数据库状态应该退回到执行事务前的状态。

C (consistency)，一致性。一致性指事务将数据库从一种状态转变为另一种一致的状态。在事务开始之前和事务结束以后，数据库的完整性约束没有被破坏。

I (isolation)，隔离性。事务的隔离性要求每个读写事务的对象与其他事务的操作对象能相互分离，即该事务提交前对其他事务都不可见，这通常使用锁来实现。

D (durability)，持久性。事务一旦提交，其结果就是永久性的，即使发生宕机等故障，数据库也能将数据恢复。持久性保证的是事务系统的高可靠性，而不是高可用性。

15、事务有哪几种类型，它们之间有什么区别？

事务可以分为以下几种类型：

扁平事务：是事务类型中最简单的一种，而在实际生产环境中，这可能是使用最为频繁的事务。在扁平事务中，所有操作都处于同一层次，其由 **BEGIN WORK** 开始，由 **COMMIT WORK** 或 **ROLLBACK WORK** 结束。处于之间的操作是原子的，要么都执行，要么都回滚。

带有保存点的扁平事务：除了支持扁平事务支持的操作外，允许在事务执行过程中回滚到同一事务中较早的一个状态，这是因为可能某些事务在执行过程中出现的错误并不会对所有的操作都无效，放弃整个事务不合乎要求，开销也太大。保存点 (savepoint) 用来通知系统应该记住事务当前的状态，以便以后发生错误时，事务能回到该状态。

链事务：可视为保存点模式的一个变种。链事务的思想是：在提交一个事务时，释放不需要的数据对象，将必要的处理上下文隐式地传给下一个要开始的事务。注意，提交事务操作和

开始下一个事务操作将合并为一个原子操作。这意味着下一个事务将看到上一个事务的结果，就好像在一个事务中进行的。

嵌套事务：是一个层次结构框架。有一个顶层事务（top-level transaction）控制着各个层次的事务。顶层事务之下嵌套的事务被称为子事务（subtransaction），其控制每一个局部的变换。

分布式事务：通常是一个在分布式环境下运行的扁平事务，因此需要根据数据所在位置访问网络中的不同节点。对于分布式事务，同样需要满足 ACID 特性，要么都发生，要么都失效。

对于 MySQL 的 InnoDB 存储引擎来说，它支持扁平事务、带有保存点的扁平事务、链事务、分布式事务。对于嵌套事务，MySQL 数据库并不是原生的，因此对于有并行事务需求的用户来说 MySQL 就无能为力了，但是用户可以通过带有保存点的事务来模拟串行的嵌套事务。

16、MySQL 的事务隔离级别

SQL 标准定义了四种隔离级别，这四种隔离级别分别是：

读未提交（READ UNCOMMITTED）；

读提交（READ COMMITTED）；

可重复读（REPEATABLE READ）；

串行化（SERIALIZABLE）。

事务隔离是为了解决脏读、不可重复读、幻读问题，下表展示了 4 种隔离级别对这三个问题的解决程度：

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED	可能	可能	可能
READ COMMITTED	不可能	可能	可能
REPEATABLE READ	不可能	不可能	可能
SERIALIZABLE	不可能	不可能	不可能

上述 4 种隔离级别 MySQL 都支持，并且 InnoDB 存储引擎默认的支持隔离级别是 REPEATABLE READ，但是与标准 SQL 不同的是，InnoDB 存储引擎在 REPEATABLE READ 事务隔离级别下，使用 Next-Key Lock 的锁算法，因此避免了幻读的产生。所以，InnoDB 存储引擎在默认的事务隔离级别下已经能完全保证事务的隔离性要求，即达到 SQL 标准的 SERIALIZABLE 隔离级别。

17、事务可以嵌套吗？

可以，因为嵌套事务也是众多事务分类中的一种，它是一个层次结构框架。有一个顶层事务控制着各个层次的事务，顶层事务之下嵌套的事务被称为子事务，它控制每一个局部的变换。需要注意的是，MySQL 数据库不支持嵌套事务。

18、MySQL 事务如何回滚？

在 MySQL 默认的配置下，事务都是自动提交和回滚的。当显式地开启一个事务时，可以使用 ROLLBACK 语句进行回滚。该语句有两种用法：

ROLLBACK: 要使用这个语句的最简形式，只需发出 ROLLBACK。同样地，也可以写为 ROLLBACK WORK，但是二者几乎是等价的。回滚会结束用户的事务，并撤销正在进行的所有未提交的修改。

ROLLBACK TO [SAVEPOINT] identifier : 这个语句与 SAVEPOINT 命令一起使用。可以把事务回滚到标记点，而不回滚在此标记点之前的任何工作。

19、数据库在什么情况下会发生死锁？

死锁是指两个或两个以上的事务在执行过程中，因争夺锁资源而造成的一种互相等待的现象。若无外力作用，事务都将无法推进下去。下图演示了死锁的一种经典的情况，即 A 等待 B、B 等待 A，这种死锁问题被称为 AB-BA 死锁。

20、对数据库优化的理解

MySQL 数据库优化是多方面的，原则是减少系统的瓶颈，减少资源的占用，增加系统的反应速度。例如，通过优化文件系统，提高磁盘 I/O 的读写速度；通过优化操作系统调度策略，提高 MySQL 在高负荷情况下的负载能力；优化表结构、索引、查询语句等使查询响应更快。

针对查询，我们可以通过使用索引、使用连接代替子查询的方式来提高查询速度。

针对慢查询，我们可以通过分析慢查询日志，来发现引起慢查询的原因，从而有针对性的进行优化。

针对插入，我们可以通过禁用索引、禁用检查等方式来提高插入速度，在插入之后再启用索引和检查。

针对数据库结构，我们可以通过将字段很多的表拆分成多张表、增加中间表、增加冗余字

段等方式进行优化。

21、介绍一下数据库设计的三大范式

目前关系数据库有六种范式，一般来说，数据库只需满足第三范式(3NF)就行了。

第一范式 (1NF)：

是指在关系模型中，对于添加的一个规范要求，所有的域都应该是原子性的，即数据库表的每一列都是不可分割的原子数据项，而不能是集合，数组，记录等非原子数据项。

即实体中的某个属性有多个值时，必须拆分为不同的属性。在符合第一范式表中的每个域值只能是实体的一个属性或一个属性的一部分。简而言之，第一范式就是无重复的域。

第二范式 (2NF)：

在 1NF 的基础上，非码属性必须完全依赖于候选码（在 1NF 基础上消除非主属性对主码的部分函数依赖）。

第二范式是在第一范式的基础上建立起来的，即满足第二范式必须先满足第一范式。第二范式要求数据库表中的每个实例或记录必须可以被唯一地区分。选取一个能区分每个实体的属性或属性组，作为实体的唯一标识。

例如在员工表中的身份证号码即可实现每个一员工的区分，该身份证号码即为候选键，任何一个候选键都可以被选作主键。在找不到候选键时，可额外增加属性以实现区分，如果在员工关系中，没有对其身份证号进行存储，而姓名可能会在数据库运行的某个时间重复，无法区分出实体时，设计譬如 ID 等不重复的编号以实现区分，被添加的编号或 ID 选作主键。

第三范式 (3NF)：

在 2NF 基础上，任何非主属性不依赖于其它非主属性（在 2NF 基础上消除传递依赖）。第三范式是第二范式的一个子集，即满足第三范式必须满足第二范式。简而言之，第三范式要求一个关系中不包含已在其它关系已包含的非主关键字信息。

例如，存在一个部门信息表，其中每个部门有部门编号 (dept_id)、部门名称、部门简介等信息。那么在员工信息表中列出部门编号后就不能再将部门名称、部门简介等与部门有关的信息再加入员工信息表中。如果不存在部门信息表，则根据第三范式 (3NF) 也应该构建它，否则就会有大量的数据冗余。

22、说一说你对 redo log、undo log、binlog 的了解

binlog (Binary Log) :

二进制日志文件就是常说的 binlog。二进制日志记录了 MySQL 所有修改数据库的操作, 然后以二进制的形式记录在日志文件中, 其中还包括每条语句所执行的时间和所消耗的资源, 以及相关的事务信息。

默认情况下, 二进制日志功能是开启的, 启动时可以重新配置--log-bin=[file_name]选项, 修改二进制日志存放的目录和文件名称。

redo log:

重做日志用来实现事务的持久性, 即事务 ACID 中的 D。它由两部分组成: 一是内存中的重做日志缓冲 (redo log buffer), 其是易失的; 二是重做日志文件 (redo log file), 它是持久的。InnoDB 是事务的存储引擎, 它通过 Force Log at Commit 机制实现事务的持久性, 即当事务提交 (COMMIT) 时, 必须先将该事务的所有日志写入到重做日志文件进行持久化, 待事务的 COMMIT 操作完成才算完成。这里的日志是指重做日志, 在 InnoDB 存储引擎中, 由两部分组成, 即 redo log 和 undo log。

redo log 用来保证事务的持久性, undo log 用来帮助事务回滚及 MVCC 的功能。redo log 基本上都是顺序写的, 在数据库运行时不需要对 redo log 的文件进行读取操作。而 undo log 是需要进行随机读写的。

undo log:

重做日志记录了事务的行为, 可以很好地通过其对页进行“重做”操作。但是事务有时还需要进行回滚操作, 这时就需要 undo。因此在对数据库进行修改时, InnoDB 存储引擎不但会产生 redo, 还会产生一定量的 undo。这样如果用户执行的事务或语句由于某种原因失败了, 又或者用户用一条 ROLLBACK 语句请求回滚, 就可以利用这些 undo 信息将数据回滚到修改之前的样子。

redo 存放在重做日志文件中, 与 redo 不同, undo 存放在数据库内部的一个特殊段 (segment) 中, 这个段称为 undo 段 (undo segment), undo 段位于共享表空间内。

23、MySQL 主从同步是如何实现的?

复制 (replication) 是 MySQL 数据库提供的一种高可用高性能的解决方案, 一般用来建立大型的应用。总体来说, replication 的工作原理分为以下 3 个步骤:

主服务器（master）把数据更改记录到二进制日志（binlog）中。

从服务器（slave）把主服务器的二进制日志复制到自己的中继日志（relay log）中。

从服务器重做中继日志中的日志，把更改应用到自己的数据库上，以达到数据的最终一致性。

复制的工作原理并不复杂，其实就是一个完全备份加上二进制日志备份的还原。不同的是这个二进制日志的还原操作基本上实时在进行中。这里特别需要注意的是，复制不是完全实时地进行同步，而是异步实时。这中间存在主从服务器之间的执行延时，如果主服务器的压力很大，则可能导致主从服务器延时较大。复制的工作原理如下图所示，其中从服务器有 2 个线程，一个是 I/O 线程，负责读取主服务器的二进制日志，并将其保存为中继日志；另一个是 SQL 线程，复制执行中继日志。



3.3.7 软件测试

1、介绍一下单元测试、集成测试、系统测试、验收测试、回归测试

1、单元测试：完成最小的软件设计单元（模块）的验证工作，目标是确保模块被正确的编码，使用过程设计描述作为指南，对重要的控制路径进行测试以发现模块内的错误，通常情况下是白盒的，对代码风格和规则、程序设计和结构、业务逻辑等进行静态测试，及早的发现和解决不易显现的错误。

2、集成测试：通过测试发现与模块接口有关的问题。目标是把通过了单元测试的模块拿来，构造一个在设计中所描述的程序结构，应当避免一次性的集成（除非软件规模很小），而采用增量集成。

自顶向下集成：模块集成的顺序是首先集成主模块，然后按照控制层次结构向下进行集成，隶属于主模块的模块按照深度优先或广度优先的方式集成到整个结构中去。

自底向上集成：从原子模块开始来进行构造和测试，因为模块是自底向上集成的，进行时要求所有隶属于某个给顶层次的模块总是存在的，也不再使用稳定测试桩的必要。

3、系统测试：是基于系统整体需求说明书的黑盒类测试，应覆盖系统所有联合的部件。系统测试是针对整个产品系统进行的测试，目的是验证系统是否满足了需求规格的定义，找出与需求规格不相符合或与之矛盾的地方。系统测试的对象不仅仅包括需要测试的产品系统的软件，还要包含软件所依赖的硬件、外设甚至包括某些数据、某些支持软件及其接口等。因此，必须将系统中的软件与各种依赖的资源结合起来，在系统实际运行环境下来进行测试。

4、回归测试：回归测试是指在发生修改之后重新测试先前的测试用例以保证修改的正确性。理论上，软件产生新版本，都需要进行回归测试，验证以前发现和修复的错误是否在新软件版本上再次出现。根据修复好了的缺陷再重新进行测试。回归测试的目的在于验证以前出现过但已经修复好的缺陷不再重新出现。一般指对某已知修正的缺陷再次围绕它原来出现时的步骤重新测试。

5、验收测试：验收测试是指系统开发生命周期方法论的一个阶段，这时相关的用户或独立测试人员根据测试计划和结果对系统进行测试和接收。它让系统用户决定是否接收系统。它是一项确定产品是否能够满足合同或用户所规定需求的测试。验收测试包括 Alpha 测试和 Beta 测试。

Alpha 测试：是由用户在开发者的场所来进行的，在一个受控的环境中进行。

Beta 测试：由软件的最终用户在一个或多个用户场所来进行的，开发者通常不在现场，用户记录测试中遇到的问题并报告给开发者，开发者对系统进行最后的修改，并开始准备发布最终的软件。

2、单元测试、集成测试、系统测试、验收测试、回归测试这几步中最重要的是哪一步

这些测试步骤分别在软件开发的阶段对软件进行测试，我认为对软件完整功能进行测试的系统测试很重要，因为此时单元测试和集成测试已完成，能够对软件所有功能进行功能测试，能够覆盖系统所有联合的部件，是针对整个产品系统进行的测试，能够验证系统是否满足了需求规格的定义，因此我认为系统测试很重要。

3、黑盒与白盒的测试方法

黑盒测试：

黑盒测试也称功能测试或数据驱动测试，它是在已知产品所应具有的功能，通过测试来检测每个功能是否都能正常使用，在测试时，把程序看作一个不能打开的黑盒子，在完全不考虑程序内部结构和内部特性的情况下，测试者在程序接口进行测试，它只检查程序功能是否按照需求规格说明书的规定正常使用，程序是否能适当地接收输入数据而产生正确的输出信息，并且保持外部信息（如数据库或文件）的完整性。

“黑盒”法着眼于程序外部结构、不考虑内部逻辑结构、针对软件界面和软件功能进行测试。“黑盒”法是穷举输入测试，只有把所有可能的输入都作为测试情况使用，才能以这种方法查出程序中所有的错误。实际上测试情况有无穷多个，因此不仅要测试所有合法的输入，而且还要对那些不合法但是可能的输入进行测试。

常用的黑盒测试方法有：等价类划分法；边界值分析法；因果图法；场景法；正交实验设计法；判定表驱动分析法；错误推测法；功能图分析法。

白盒测试：

白盒测试也称为结构测试或逻辑驱动测试，是针对被测单元内部是如何进行工作的测试。它根据程序的控制结构设计测试用例，主要用于软件或程序验证。白盒测试法检查程序内部逻辑结构，对所有的逻辑路径进行测试，是一种穷举路径的测试方法，但即使每条路径都测试过了，但仍然有可能存在错误。因为：穷举路径测试无法检查出程序本身是否违反了设计规范，即程序是否是一个错误的程序；穷举路径测试不可能检查出程序因为遗漏路径而出错；穷举路

径测试发现不了与数据相关的错误。

白盒测试需要遵循的原则有：1. 保证一个模块中的所有独立路径至少被测试一次；2. 所有逻辑值均需要测试真（true）和假（false）；两种情况；3. 检查程序的内部数据结构，保证其结构的有效性；4. 在上下边界及可操作范围内运行所有循环。

4、手动测试与自动化测试的优缺点

手工测试缺点：

- 1、重复的手工回归测试，代价昂贵、容易出错。
- 2、依赖于软件测试人员的能力。

手工测试优点：

- 1、测试人员具有经验和对错误的猜测能力。
- 2、测试人员具有审美能力和心理体验。
- 3、测试人员具有是非判断和逻辑推理能力。

自动化测试的优点：

1、对程序的回归测试更方便。这可能是自动化测试最主要的任务，特别是在程序修改比较频繁时，效果是非常明显的。由于回归测试的动作和用例是完全设计好的，测试期望的结果也是完全可以预料的，将回归测试自动运行，可以极大提高测试效率，缩短回归测试时间。

2、可以运行更多更繁琐的测试。自动化的一个明显的好处是可以在较少的时间内运行更多的测试。

3、可以执行一些手工测试困难或不可能进行的测试。比如，对于大量用户的测试，不可能同时让足够多的测试人员同时进行测试，但是却可以通过自动化测试模拟同时有许多用户，从而达到测试的目的。

4、更好地利用资源。将繁琐的任务自动化，可以提高准确性和测试人员的积极性，将测试技术人员解脱出来投入更多精力设计更好的测试用例。有些测试不适合于自动测试，仅适合于手工测试，将可自动测试的测试自动化后，可以让测试人员专注于手工测试部分，提高手工测试的效率。

5、测试具有一致性和可重复性。由于测试是自动执行的，每次测试的结果和执行的内容的一致性是可以得到保障的，从而达到测试的可重复的效果。

6、测试的复用性。由于自动测试通常采用脚本技术，这样就有可能只需要做少量的甚至

不做修改，实现在不同的测试过程中使用相同的用例。

7、增加软件信任度。由于测试是自动执行的，所以不存在执行过程中的疏忽和错误，完全取决于测试的设计质量。一旦软件通过了强有力的自动测试后，软件的信任度自然会增加。

自动化测试的缺点：

1、不能取代手工测试

2、手工测试比自动测试发现的缺陷更多

3、对测试质量的依赖性极大

4、测试自动化不能提高有效性

5、测试自动化可能会制约软件开发。由于自动测试比手动测试更脆弱，所以维护会受到限制，从而制约软件的开发。

6、工具本身并无想像力

5、你觉得软件测试的核心竞争力是什么

测试人员的核心竞争力在于提早发现问题，并能够发现别人无法发现的问题。

1、早发现问题：问题发现的越早，解决的成本越低。如果一个需求在还未实现的时候就能发现需求的漏洞，那么这种问题的价值是最高的。

2、发现别人无法发现的问题：所有人都能发现的问题，你发现了，那就证明你是可以被替代的。别人发现不了，而你可以发现，那么你就是无法被替代。

6、如何写测试用例

1、测试人员尽早介入，彻底理解清楚需求，这个是写好测试用例的基础

2、如果以前有类似的需求，可以参考类似需求的测试用例，然后还需要看类似需求的 bug 情况

3、清楚输入、输出的各种可能性，以及各种输入的之间的关联关系，理解清楚需求的执行逻辑，通过等价类、边界值、判定表等方法找出大部分用例

4、找到需求相关的一些特性，补充测试用例

5、根据自己的经验分析遗漏的测试场景

6、多总结类似功能点的测试点，才能够写出质量越来越高的测试用例

7、书写格式一定要清晰

7、请你说一说测试用例的边界

边界值分析法就是对输入或输出的边界值进行测试的一种黑盒测试方法。通常边界值分析法是作为对等价类划分法的补充, 这种情况下, 其测试用例来自等价类的边界。

常见的边界值

- 1)对 16-bit 的整数而言 32767 和 -32768 是边界
- 2)屏幕上光标在最左上、最右下位置
- 3)报表的第一行和最后一行
- 4)数组元素的第一个和最后一个
- 5)循环的第 0 次、第 1 次和倒数第 2 次、最后一次

8、软件质量的六个特征

按照软件质量国家标准 GB-T8566--2001G, 软件质量可以用下列特征来评价:

- a.功能特征: 与一组功能及其指定性质有关的一组属性, 这里的功能是满足明确或隐含的需求的那些功能。
- b.可靠特征: 在规定的一段时间和条件下, 与软件维持其性能水平的能力有关的一组属性。
- c.易用特征: 由一组规定或潜在的用户为使用软件所需作的努力和所作的评价有关的一组属性。
- d.效率特征: 与在规定条件下软件的性能水平与所使用资源量之间关系有关的一组属性。
- e.可维护特征: 与进行指定的修改所需的努力有关的一组属性。
- f.可移植特征: 与软件从一个环境转移到另一个环境的能力有关的一组属性。

9、黑盒测试和白盒测试有哪些方法

黑盒测试方法有等价类划分, 边界值分析, 错误推测, 因果图法

白盒测试方法有逻辑覆盖法, 程序插桩技术, 基本路径法, 符号测试, 错误驱动测试

10、测试方法有哪些?

等价类划分, 边界值分析, 错误推测, 因果图法, 逻辑覆盖法, 程序插桩技术, 基本路径法, 符号测试, 错误驱动测试

3.3.8 机器学习

1、循环神经网络，为什么好？

循环神经网络模型（RNN）是一种节点定向连接成环的人工神经网络，是一种反馈神经网络，RNN 利用内部的记忆来处理任意时序的输入序列，并且在其处理单元之间既有内部的反馈连接又有前馈连接，这使得 RNN 可以更加容易处理不分段的文本等。

2、什么是 Group Convolution

若卷积神经网络的上层有 N 个卷积核,则对应的通道数也为 N 。设群数目为 M ,在进行卷积操作的时候,将通道分成 M 份,每个 group 对应 N/M 个通道,然后每个 group 卷积完成后输出叠在一起,作为当前层的输出通道。

3、什么是 RNN

一个序列当前的输出与前面的输出也有关,在 RNN 网络结构中,隐藏层的输入不仅包括输入层的输出还包含上一时刻隐藏层的输出,网络会对之前的信息进行记忆并应用于当前的输入计算中。

4、训练过程中, 若一个模型不收敛, 那么是否说明这个模型无效?导致模型不收敛的原因有哪些?

并不能说明这个模型无效,导致模型不收敛的原因可能有数据分类的标注不准确,样本的信息量太大导致模型不足以 fit 整个样本空间。学习率设置的太大容易产生震荡,太小会导致不收敛。可能复杂的分类任务用了简单的模型。数据没有进行归一化的操作。

5、图像处理中锐化和平滑的操作

锐化就是通过增强高频分量来减少图像中的模糊,在增强图像边缘的同时也增加了图像的噪声。

平滑与锐化相反,过滤掉高频分量,减少图像的噪声是图片变得模糊。

6、Attention 机制的作用

减少处理高维输入数据的计算负担,结构化的选取输入的子集,从而降低数据的维度。让系统更加容易的找到输入的数据中与当前输出信息相关的有用信息,从而提高输出的质量。帮助类似于 **decoder** 这样的模型框架更好的学到多种内容模态之间的相互关系。

7、Lstm 和 Gru 的原理

Lstm 由输入门,遗忘门,输出门和一个 **cell** 组成。第一步是决定从 **cell** 状态中丢弃什么信息,然后在决定有多少新的信息进入到 **cell** 状态中,最终基于目前的 **cell** 状态决定输出什么样的信息。

Gru 由重置门和更新门组成,其输入为前一时刻隐藏层的输出和当前的输入,输出为下一时刻隐藏层的信息。重置门用来计算候选隐藏层的输出,其作用是控制保留多少前一时刻的隐藏层。更新门的作用是控制加入多少候选隐藏层的输出信息,从而得到当前隐藏层的输出。

8、什么是 dropout

在神经网络的训练过程中,对于神经单元按一定的概率将其随机从网络中丢弃,从而达到对于每个 **mini-batch** 都是在训练不同网络的效果,防止过拟合。

9、BN 层的作用，为什么要在后面加伽马和贝塔，不加可以吗

BN 层的作用是把一个 **batch** 内的所有数据，从不规范的分布拉到正态分布。这样做的好处是使得数据能够分布在激活函数的敏感区域，敏感区域即为梯度较大的区域，因此在反向传播的时候能够较快反馈误差传播。

10、Adam

Adam 算法和传统的随机梯度下降不同。随机梯度下降保持单一的学习率（即 **alpha**）更新所有的权重，学习率在训练过程中并不会改变。而 **Adam** 通过计算梯度的一阶矩估计和二阶矩估计而为不同的参数设计独立的自适应性学习率。

11、GAN 网络的思想

GAN 用一个生成模型和一个判别模型,判别模型用于判断给定的图片是不是真实的图片,生成模型自己生成一张图片和想要的图片很像,开始时两个模型都没有训练,然后两个模型一起进

行对抗训练,生成模型产生图片去欺骗判别模型,判别模型去判别真假,最终两个模型在训练过程中,能力越来越强最终达到稳态。

12、怎么提升网络的泛化能力

从数据上提升性能:收集更多的数据,对数据做缩放和变换,特征组合和重新定义问题。

从算法调优上提升性能:用可靠的模型诊断工具对模型进行诊断,权重的初始化,用小的随机数初始化权重。对学习率进行调节,尝试选择合适的激活函数,调整网络的拓扑结构,调节 batch 和 epoch 的大小,添加正则化的方法,尝试使用其它的优化方法,使用 early stopping。

13、激活函数的作用

激活函数是用来加入非线性因素的,提高神经网络对模型的表达能力,解决线性模型所不能解决的问题。

14、神经网络为啥用交叉熵。

通过神经网络解决多分类问题时,最常用的一种方式就是在最后一层设置 n 个输出节点,无论在浅层神经网络还是在 CNN 中都是如此,比如,在 AlexNet 中最后的输出层有 1000 个节点,而即便是 ResNet 取消了全连接层,也会在最后有一个 1000 个节点的输出层。

一般情况下,最后一个输出层的节点个数与分类任务的目标数相等。假设最后的节点数为 N ,那么对于每一个样例,神经网络可以得到一个 N 维的数组作为输出结果,数组中每一个维度会对应一个类别。在最理想的情况下,如果一个样本属于 k ,那么这个类别所对应的的输出节点的输出值应该为 1,而其他节点的输出都为 0,即 $[0,0,1,0,\dots,0,0]$,这个数组也就是样本的 Label,是神经网络最期望的输出结果,交叉熵就是用来判定实际的输出与期望的输出的接近程度。

15、CNN 为什么比 DNN 在图像识别上更好

DNN 的输入是向量形式,并未考虑到平面的结构信息,在图像和 NLP 领域这一结构信息尤为重要,例如识别图像中的数字,同一数字与所在位置无关(换句话说任一位置的权重都应相同),CNN 的输入可以是 tensor,例如二维矩阵,通过 filter 获得局部特征,较好的保留了平面结构信息。

3.3.9 设计模式

1、设计模式的六大原则

单一职责原则、里氏替换原则、依赖倒置原则、接口隔离原则、迪米特法则、开闭原则。

2、说一下六大原则中的开闭原则

开闭原则的定义是：一个软件实体应当对扩展开放，对修改关闭。这个原则说的是，在设计一个模块的时候，应当使这个模块可以在不被修改的前提下被扩展，即应当可以在不必修改源代码的情况下改变这个模块的行为。

在面向对象的编程中，开闭原则是最基础的原则，起到总的指导作用，其他原则（单一职责、里氏替换、依赖倒置、接口隔离、迪米特法则）都是开闭原则的具体形态，即其他原则都是开闭原则的手段和工具。开闭原则的重要性可以通过以下几个方面来体现。

开闭原则提高复用性。在面向对象的设计中，所有的逻辑都是从原子逻辑组合而来的，而不是在一个类中独立实现一个业务逻辑，代码粒度越小，被复用的可能性就越大，避免相同的逻辑重复增加。开闭原则的设计保证系统是一个在高层次上实现了复用的系统。

开闭原则提高可维护性。一个软件投产后，维护人员的工作不仅仅是对数据进行维护，还可能对程序进行扩展，就是扩展一个类，而不是修改一个类。开闭原则对已有软件模块，特别是最重要的抽象层模块要求不能再修改，这就使变化中的软件系统有一定的稳定性和延续性，便于系统的维护。

开闭原则提高灵活性。所有的软件系统都有一个共同的性质，即对系统的需求都会随时间的推移而发生变化。在软件系统面临新的需求时，系统的设计必须是稳定的。开闭原则可以通过扩展已有的软件系统，提供新的行为，能快速应对变化，以满足对软件新的需求，使变化中的软件系统有一定的适应性和灵活性。

开闭原则易于测试。测试是软件开发过程中必不可少的一个环节。测试代码不仅要保证逻辑的正确性，还要保证苛刻条件（高压、异常、错误）下不产生“有毒代码”（Poisonous Code），因此当有变化提出时，原有健壮的代码要尽量不修改，而是通过扩展来实现。否则，就需要把原有的测试过程回笼一遍，需要进行单元测试、功能测试、集成测试，甚至是验收测试。开闭原则的使用，保证软件是通过扩展来实现业务逻辑的变化，而不是修改。因此，对于新增加类，只需新增相应的测试类，编写对应的测试方法，只要保证新增的类是正确的就可以了。

3、说一说你对工厂模式的理解

工厂模式的用意是定义一个创建产品对象的工厂接口，将实际创建性工作推迟到子类中。工厂模式可分为简单工厂、工厂方法和抽象工厂模式。注意，我们常说的 23 种经典设计模式，包含了工程方法模式和抽象工厂模式，而并未包含简单工厂模式。另外，我们平时说的工厂模式，一般默认是指工厂方法模式。

4、简单工厂模式和抽象工厂模式有什么区别？

简单工厂模式其实并不算是一种设计模式，更多的时候是一种编程习惯。简单工厂的实现思路是，定义一个工厂类，根据传入的参数不同返回不同的实例，被创建的实例具有共同的父类或接口。

工厂方法模式是简单工厂的仅一步深化，在工厂方法模式中，我们不再提供一个统一的工厂类来创建所有的对象，而是针对不同的对象提供不同的工厂。也就是说每个对象都有一个与之对应的工厂。工厂方法的实现思路是，定义一个用于创建对象的接口，让子类决定将哪一个类实例化。工厂方法模式让一个类的实例化延迟到其子类。

抽象工厂模式是工厂方法的仅一步深化，在这个模式中的工厂类不单单可以创建一个对象，而是可以创建一组对象。这是和工厂方法最大的不同点。抽象工厂的实现思路是，提供一个创建一系列相关或相互依赖对象的接口，而无须指定它们具体的类。

第四章 常见问题答疑

4.1 考研流程



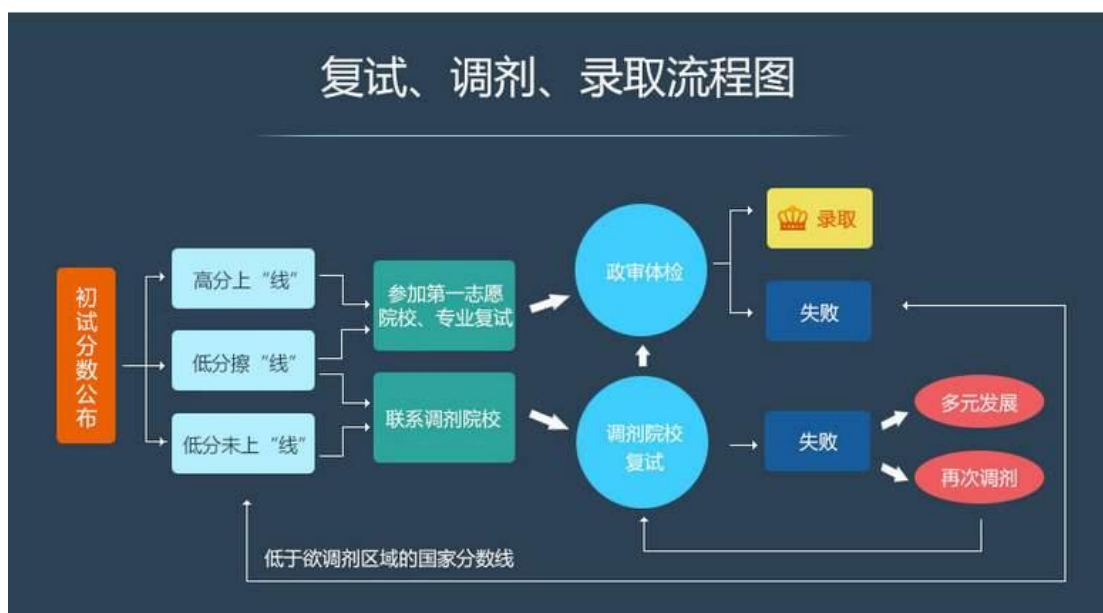
具体介绍

1. 考研目标确定：目标分为院校和专业。考生确定考研院校和专业，这是考研关键的一步。考生根据自身实力及人生规划，确定报考学校及专业。考生确定院校及专业后，能够找准复习方向，有针对性的进行考研准备。
2. 复习计划制定：考生在复习开始前，需要制定一份完整的、详细的复习计划。考生严格执行考研复习计划，能够提高复习效率、避免在后期的复习过程中出现复习进度安排不恰当的情况。
3. 考研基础复习：基础阶段的复习需要考生静下心来，不要因为一到两道题目不会而心烦意乱，掌握做题方法，基础牢固即可。
4. 暑期强化复习：暑期是考研复习中的黄金时间段。通常很多同学会选择利用暑期时间报名参加各类辅导班，以弥补个人复习过程中的不足。这里建议大家学会在 B 站搜索相关的考研视频即可，或者加入 N 诺的交流群与大家交流分享最新的考研资料。
5. 招生简章发布：各院校开始公布招生简章。招生简章包括：学校基本情况介绍、热门专业及院校介绍、报考条件、录取政策、历年报录数据、常见问题汇总、联系方式、收费标准等针对本单位的详细信息，以便考生及时了解招生政策及安排。考生一定要阅读目标院校的招生简章，与前一年招生简章相比发生了哪些变化，这些变化是否对你有影响。
6. 考研大纲发布：9 月份，教育部发布统考公共课及统考专业课大纲。在各科考试大纲中，明确相应科目的试题结构。考生可根据考研大纲适当调整自己的复习计划。
7. 考研网上报名：网上预报名、全国统一报名。网上预报名是为了让应届生了解考研报名过程，缓冲报名当天的网站荷载。网上预报名的结果可以更改。
8. 考研报名确认：核对考生基本信息、采集图像、考试费用缴纳等。
9. 考研冲刺复习：考生结合自身复习进度，按照考试时间进行模拟测试，查漏补缺。
10. 准考证打印：考生凭网报用户名和密码登录中国研究生招生信息网下载打印《准考证》。在考试结束前，均可以进行准考证的打印。考生可多打印几份准考证备用，以防丢失。
11. 初试：考生在考试前仔细确认考场安排，妥善安排出行，不在本学校考试的考生可以提前在考场旁边的酒店等地方租房，避免来回路上耽误时间。

12. 成绩查询及复查：考研初试成绩一般在考 2 月公布，如果对个人成绩有异议，可以按照招生单位统一安排申请复查。
13. 自主划线学校公布分数线：34 所自主划线学校结合本校招生计划及报考情况，自行确定复试基本分数线并公布。
14. 研究生入学分数线公布：分数线一般在 3 月中旬公布，教育部会结合当年招生计划及报考情况，分别确定一、二区考生进入复试的基本分数线。
15. 考研复试：非自划线学校在分数线公布之后，结合本校招生计划，在第 1 时间确定本校复试基本分数线，陆续安排复试。
16. 考研调剂：考研调剂越早报名越好。
17. 领取通知书：学校一般在 6 月初开始陆续发放通知书。
18. 入学：按照学校安排，统一办理入学。



4.2 复试流程



如何确定是否能进复试？

- 1、四门课是否过了单科线
- 2、总分是否过了总分线
- 3、按比例确定复试名单

如何计算综合成绩？

最终成绩 = 初试成绩 * 初试权重 + 复试成绩 * 复试权重

注：每所院校权重比例不同，参考往年复试通知

例如：

某同学初试 380 分，复试 86 分，目标院校的复试权重占 30%。

那么该同学的最终成绩为： $(380 / 5) * 0.7 + 86 * 0.3 = 79$ 分（百分制）

4.3 复试是否需要报培训班?

先说结论：不需要

先从淘汰率来看, 对于大多数学校来说差额复试比例一般是 1:1.2。也就是相对于初试来说, 淘汰率比较低。除了有很少部分全日制和非全日制统一划线复试录取的学校, 是比较难受的, 其他的学校只要初试分数较高都是很稳的。

对于初试分数排名前 50% 的同学们来说, 除了机试环节可能分数差距过大被翻盘, 其他环节很难拉开多大的差距, 只要认真准备了, 分数差距都很小。

对于初试分数排名后 50% 的同学来说, 每个环节都重要, 特别是刚过线进复试的同学, 尽量让自己每个环节都不要有短板才能翻盘。

其实报培训班最大的好处是可以锻炼表达能力, 对着空气说和对着人说的感觉是不一样的, 有的同学性格内向容易紧张。但是花费几千甚至上万参加培训班是不值当的, 建议同学们可以找一个同学互相模拟面试就可以。

最后, 祝每一个同学都能顺利上岸~

N 诺考研系列图书

《计算机考研报考指南》

《计算机考研复试高分指南》

《C 语言考研复习攻略》

《数据结构考研复习攻略》

《操作系统考研复习攻略》

《计算机网络考研复习攻略》

《计算机组成原理考研复习攻略》

《数据库考研复习攻略》

《计算机考研机试攻略 - 高分篇》

《计算机考研机试攻略 - 满分篇》

考研路上，N 诺与你携手同行。